

Compiling Command Interfaces

A Native Typed CLI Runtime for Glazed: Schema Compilation, Residual Parsing,
Provenance, and Modern Help

A design and experimental thesis based on the Glazed codebase

2026-08-16

Contents

Abstract	1
Preface	2
Repository snapshot and evidence discipline	2
The running example: golem	3
1 Glazed as a Latent Language Runtime	4
1.1 Learning objectives	4
1.2 Motivation: the dependency is not the architecture	4
1.3 Research questions	5
1.4 The current architecture	6
1.5 Command descriptions as language declarations	6
1.5.1 Worked example: declaring note search	7
1.5.2 Worked example: the same command through a non-CLI frontend	7
1.6 Sections and field definitions form a typed signature	8
1.6.1 Worked example: two sections, one semantic namespace	8
1.7 Semantic codecs: the parser should not own types	9
1.7.1 Worked example: three surface forms, one codec input	9
1.7.2 Counterexample: parser-specific typed storage	9
1.8 Source middleware and provenance are already a resolution algebra	10
1.8.1 Worked example: explain a winning value	10
1.9 The frontend-neutral runner closes the semantic loop	10
1.10 Aliases are macros, not merely alternate names	11
1.10.1 Counterexample: copying the target schema at alias construction	11
1.11 The help system is modern in content, legacy in its command adapter	11
1.12 Historical repairs as design evidence	12
1.13 Why the existing string-list parser is a seed, not the final design	12
1.14 Chapter synthesis	13
1.15 Exercises for Chapter 1	13
2 A Mathematical Semantics for Typed Command Lines	15
2.1 Learning objectives	15
2.2 Motivation: compatibility requires a language specification	15
2.3 Command paths form a prefix language	16
2.3.1 Worked example: path resolution	16
2.3.2 Counterexample: guessing namespaces at runtime	16
2.4 Semantic fields and frontend bindings	16
2.4.1 Worked example: prefixed sections	19
2.4.2 Worked example: deprecated spellings	19
2.4.3 Counterexample: string keys as semantic identity	19
2.5 Whole-program compilation	19
2.5.1 Compile-time obligations	20
2.5.2 Worked example: atomic collision reporting	21

2.6	Positional arguments form a small grammar	21
2.6.1	Worked example 2.5	21
2.6.2	Counterexample: positional parsing before command selection	22
2.7	A deterministic scanning transition system	22
2.7.1	Core transition rules	23
2.7.2	Worked example: tracing an alias and a cluster	23
2.7.3	Worked example: negative numbers	23
2.7.4	Counterexample: Boolean options that optionally consume a value	24
2.8	Raw occurrences precede repeat policy	24
2.8.1	Worked example: why two policies are needed	24
2.9	Source precedence as ordered partial-map override	25
2.9.1	Provenance-enriched fold	26
2.9.2	Worked example: requiredness after merge	26
2.9.3	Counterexample: pflag-level required markers	26
2.10	Defaults, absence, null, and empty values	26
2.10.1	Worked example: empty list overrides a default	27
2.11	Aliases as a rewrite system	27
2.11.1	Worked example: precedence of injected and caller options	27
2.11.2	Counterexample: textual expansion after parsing	27
2.12	Completion is parsing the residual language	27
2.12.1	Worked example: completing an option name	28
2.12.2	Worked example: completing a choice value	28
2.12.3	Worked example: dynamic completion	28
2.12.4	Counterexample: completion that calls normal execution	29
2.13	Structured diagnostics	29
2.13.1	Diagnostic taxonomy	29
2.14	POSIX compatibility and intentional language choices	30
2.15	Formal properties worth testing	30
2.16	Chapter synthesis	31
2.17	Exercises for Chapter 2	32
3	Architecture and Experimental Implementation	33
3.1	Learning objectives	33
3.2	Motivation: experiment on the architecture, not only the scanner	33
3.3	Target architecture	34
3.4	Proposed package decomposition	34
3.4.1	pkg/cmds/model	35
3.4.2	pkg/cmds/catalog	35
3.4.3	pkg/cmds/fields	35
3.4.4	pkg/cli/native	35
3.4.5	pkg/cmds/sources	35
3.4.6	pkg/cli/complete	35
3.4.7	pkg/help/core	35
3.4.8	compat/cobra	36
3.5	Catalog API design	36
3.6	Field bindings as compiled evidence	37
3.6.1	Worked example: detecting collisions before command mount	37
3.7	Parse API design	38
3.7.1	Scanner and semantic decoder boundary	38
3.8	Resolution API design	38
3.9	Command contracts	39
3.9.1	Worked example: describe versus plan	39
3.10	Help as a typed intermediate representation	40
3.11	Joining reference help and authored help	41
3.11.1	Worked example: contextual error help	41
3.12	The experimental prototype	42

3.12.1	Implemented features	42
3.12.2	Representative authored fixture	42
3.12.3	Representative resolution	43
3.13	Experimental questions	43
3.14	Validation results	43
3.15	Microbenchmark results	44
3.15.1	Interpretation	45
3.16	Static audit of the existing native parser seed	45
3.17	Compatibility matrix	45
3.18	Differential and metamorphic testing	46
3.18.1	Differential testing	46
3.18.2	Metamorphic properties	47
3.18.3	Golden compatibility corpus	47
3.19	Prototype limitations	47
3.20	Refinement obligations	48
3.21	Chapter synthesis	48
3.22	Exercises for Chapter 3	49
4	Migration, Modern Help, and Framework Evolution	50
4.1	Learning objectives	50
4.2	Motivation: dependency removal is a public-language migration	50
4.3	A seven-phase migration	51
4.3.1	Phase 0 - Freeze and observe current semantics	51
4.3.2	Phase 1 - Introduce a pure catalog compiler	52
4.3.3	Phase 2 - Add the native scanner and resolver	52
4.3.4	Phase 3 - Add native completion and help reference rendering	52
4.3.5	Phase 4 - Migrate first-party applications	52
4.3.6	Phase 5 - Isolate the Cobra adapter	53
4.3.7	Phase 6 - Publish the native major release	53
4.4	Compatibility profiles	53
4.4.1	Worked example: a deliberate break	53
4.4.2	Worked example: command-prefix matching	54
4.5	Differential testing as the migration gate	54
4.5.1	Canonicalization rules	54
4.5.2	Evidence gates	54
4.6	Fuzzing and property testing	55
4.6.1	Catalog fuzzing	55
4.6.2	Invocation fuzzing	55
4.6.3	Generative valid programs	55
4.7	Rearchitecting the Command framework	55
4.7.1	Layer 1 - authoring declarations	55
4.7.2	Layer 2 - compiled static contract	55
4.7.3	Layer 3 - invocation interpretation	56
4.7.4	Layer 4 - effects	56
4.7.5	Proposed top-level API	56
4.8	Command sets and catalogs	56
4.8.1	Worked example: plugin reload	57
4.9	Framework-injected inputs	57
4.10	Constraints beyond requiredness	57
4.10.1	Worked example: cache clear	58
4.11	Modern help: from command template to help service	58
4.11.1	help - what can I do?	58
4.11.2	describe - what is the static contract?	58
4.11.3	explain - why did this invocation resolve this way?	58
4.11.4	plan - what would execute?	59
4.12	Help rendering modes	59

4.12.1	Concise terminal view	59
4.12.2	Long contextual view	59
4.12.3	Searchable TUI	59
4.12.4	JSON view	60
4.12.5	Web documentation	60
4.13	Contextual help and diagnostics	60
4.14	Help content governance	60
4.15	Shell completion protocol	60
4.15.1	Completion safety policy	61
4.16	Error and exit semantics	61
4.16.1	Exit taxonomy	61
4.17	Secrets and security boundaries	62
4.17.1	Secret field policy	62
4.17.2	Authorization	62
4.18	Downstream compatibility and semantic versioning	62
4.18.1	Application authors using only Glazed APIs	63
4.18.2	Application authors importing Cobra through Glazed	63
4.18.3	End users and scripts	63
4.19	Migration tooling	63
4.19.1	glazed doctor cli	63
4.19.2	glazed manifest diff	63
4.19.3	glazed cli replay	64
4.19.4	glazed help lint	64
4.20	Governance of the native language	64
4.21	Future research	64
4.21.1	Formal grammar extraction	64
4.21.2	Constraint solving	64
4.21.3	Incremental catalog compilation	65
4.21.4	Editor and agent protocols	65
4.21.5	Effect contracts and policy	65
4.21.6	Documentation verification	65
4.21.7	Reproducible command manifests	65
4.22	Final conclusions	65
4.23	Exercises for Chapter 4	66
Selected Solutions and Discussion		67
4.24	Solution 1.1: semantic versus frontend concerns	67
4.25	Solution 1.4: two format fields	67
4.26	Solution 2.3: why injectivity is necessary	67
4.27	Solution 2.7: override is not commutative	67
4.28	Solution 2.11: completion after --	67
4.29	Solution 3.5: alias source maps	67
4.30	Solution 3.10: help with missing required fields	68
4.31	Solution 4.4: dual-run without duplicate effects	68
4.32	Solution 4.7: exactly-one formula	68
4.33	Solution 4.10: binding rename	68
Glossary and Notation		69
Appendix A: Prototype API and Reproduction		72
Appendix B: Migration Checklist		73
4.34	Semantic inventory	73
4.35	Grammar	73
4.36	Compiler	73
4.37	Runtime	73

4.38Evidence	74
Appendix C: Compatibility Questions That Must Be Answered	75
Appendix D: Glazed Source Evidence Map	76
Bibliography	77

Abstract

Glazed is commonly described as a Go framework for command-line applications that emit structured data. That description is correct but incomplete. The current codebase already contains most of the components of a small typed language runtime: commands have stable descriptions; inputs are organized into ordered sections; fields have semantic types, defaults, choices, requiredness, and positional roles; value sources are composed through precedence-aware middleware; resolved values carry provenance logs; and effects are executed through a frontend-neutral runner. Cobra and pflag remain embedded at the outer boundary, where they currently supply command-tree mounting, token scanning, flag storage, portions of command resolution, shell-completion integration, and the shape expected by the help adapter.

This thesis asks whether Glazed can remove that dependency without discarding the semantic model that makes Glazed valuable. It argues that the correct unit of replacement is not the flag parser alone. The correct unit is a **compiled command language**: a whole-program transformation from mutable command definitions into an immutable catalog used by parsing, completion, help, machine-readable description, source resolution, and execution. The central design principle is that semantic field identity must be separated from frontend bindings. A field such as `search.limit` is not the string `--limit`; it is a typed semantic slot that may have CLI, environment, configuration, JSON, and HTTP bindings.

The thesis makes four contributions. First, it reconstructs the pinned Glazed repository at commit `a2bff0ece5f46b90975d7687f7c0dca2ea516d22` as a latent language runtime and maps the exact Cobra coupling. Second, it develops a mathematical model of command paths, effective scopes, option bindings, source precedence, provenance, aliases, diagnostics, and completion. In that model, completion is the residual language after a typed prefix, and source resolution is a deterministic fold over ordered partial maps. Third, it reports an executable prototype that compiles an immutable catalog, parses long and clustered short options, supports `--`, expands aliases, resolves typed values with source histories, validates required fields after all sources merge, derives completion from partial parser state, and renders help and manifests from the same catalog. The prototype passes race-enabled tests with 73.6 percent statement coverage in its core package. On the supplied synthetic workload, compiling 100 commands with 40 options each takes a median of approximately 3.41 ms; parsing a representative invocation takes approximately 1.57 microseconds; and completing a partial invocation takes approximately 5.86 microseconds. These measurements are feasibility evidence, not production performance claims.

Fourth, the thesis presents a staged migration plan and a modern help architecture. The proposed help system combines exact reference facts from the compiled catalog with Glazed's existing authored Markdown corpus and searchable Bubble Tea interface. It distinguishes help, describe, explain, and plan: reference rendering, static machine contract, provenance-aware interpretation, and value-dependent execution planning are related but non-equivalent operations.

The conclusion is qualified. Removing Cobra is feasible and architecturally coherent, but a compatible replacement is a language migration, not a dependency edit. Success requires an explicit grammar, whole-program compilation, structured diagnostics, differential tests, and a major-version policy for intentional syntax changes. The proposed architecture preserves Glazed's strongest existing ideas while moving command routing, parsing, completion, and help under one native semantic authority.

Preface

A command-line interface is often introduced as a thin shell around a program. That description works for a single command with three options. It fails for a framework such as Glazed. Glazed commands can be loaded from Go or declarative sources, organized into command paths, composed with reusable field sections, populated from defaults and configuration overlays, overridden by environment variables and flags, decoded into typed values, inspected through provenance logs, rendered through several output modes, and documented through a searchable help corpus. Once these features interact, the CLI is no longer a shell. It is a language implementation.

The immediate engineering goal behind this study is concrete: remove the dependency on github.com/spf13/cobra, implement native flag parsing, reconsider the command framework and its mappings to CLI syntax, and modernize help where the existing architecture permits. The pedagogical goal is broader. A parser replacement should be understandable in the language of computer science: grammars, tries, partial functions, typed signatures, transition systems, algebraic composition, residual languages, interpreters, proof obligations, and refinement tests. Those concepts are not decoration. They are tools for avoiding a replacement that merely reproduces Cobra's accidental constraints under a different package name.

This document is written as a doctoral-style design thesis rather than a source-code tour. Each important term is introduced through four steps:

1. **motivation** - the concrete engineering problem that forces the concept to exist;
2. **definition** - a precise statement, often mathematical or API-level;
3. **worked examples** - several applications to a continuous running example;
4. **counterexample and exercises** - a failure mode that tests the boundary of the definition, followed by problems for the reader.

The thesis does not claim that the experimental prototype is production-ready. It is intentionally small, uses only the Go standard library, and omits several compatibility features that a final implementation must decide deliberately. Its purpose is to test the architecture, expose hidden contracts, and make the migration plan falsifiable.

Repository snapshot and evidence discipline

The source analysis is pinned to Glazed commit `a2bff0ece5f46b90975d7687f7c0dca2ea516d22` on the main branch. Runtime code and tests are treated as stronger evidence than comments or documentation. Historical pull requests are used to explain design pressure, but the pinned source is the authority for current behavior. In particular, older discussions of a forty-four-option structured-output surface are historical: current Glazed has already reduced the default output surface to `--format`, `--output-fields`, and `--max-output-rows`.

The most important inspected components are:

- `pkg/cmds`: command interfaces and `CommandDescription`;
- `pkg/cmds/schema`: ordered sections and schema composition;
- `pkg/cmds/fields`: field definitions, semantic codecs, positional parsing, the Cobra bridge, and the existing string-list parser;

- pkg/cmds/sources: source middleware and precedence;
- pkg/cmds/values: resolved section values and parse histories;
- pkg/cmds/runner: frontend-neutral effect execution;
- pkg/cli: Cobra construction and parser integration;
- pkg/help: authored help sections, query, terminal rendering, and the Bubble Tea browser;
- pkg/cmds/alias: aliases as command definitions with fixed arguments and flags.

A detailed source map appears in Appendix D.

The running example: golem

One fictional application is used throughout the thesis:

```
golem [--config FILE] [--verbose]
note
  search QUERY [--limit N] [--tag TAG]... [--format FORMAT]
  show SLUG
cache
  clear [--all | --key KEY]
find QUERY ...           # alias for note search --format=jsonl
```

`golem` resembles applications already built with `Glazed`: it has a command hierarchy, structured output, reusable configuration, aliases, positional arguments, list-valued options, and a help corpus. The example is deliberately small enough to calculate by hand and rich enough to expose the difficult cases.

Notation. Finite token sequences are written as $\langle t_1, \dots, t_n \rangle$. A partial function is written $f : A \rightharpoonup B$. A finite partial map from keys to values is written $M : K \rightharpoonup V$. The right-biased override of two partial maps is written $A \triangleright B$, meaning that B wins wherever it is defined. The path of a command node v is $\text{path}(v)$.

Chapter 1

Glazed as a Latent Language Runtime

1.1 Learning objectives

After this chapter, the reader should be able to:

- distinguish Glazed’s semantic command model from its Cobra frontend;
- explain why replacing pflag alone is insufficient;
- identify the frontend-neutral components that should survive the migration;
- describe field sections, semantic codecs, source middleware, provenance, aliases, and the runner as parts of one language runtime;
- state the thesis research questions and evaluate the proposed contributions against them.

1.2 Motivation: the dependency is not the architecture

Suppose a maintainer begins with the following plan:

1. delete the Cobra import;
2. scan `os.Args` manually;
3. put values into `values.Values`;
4. keep the rest unchanged.

The plan sounds local because flag parsing sounds local. It fails as soon as the maintainer asks what Cobra currently does on Glazed’s behalf. Cobra is not only a scanner. The adapter creates command nodes, synthesizes parent namespaces, attaches pflag values, decides which command was selected, hands positional arguments to Glazed, carries context and I/O, provides the API shape consumed by the help integration, and participates in shell completion. The current `schema.Section` interface even contains Cobra-specific methods, so the dependency reaches into what otherwise appears to be the core schema package.

The opposite mistake is equally damaging: replacing all of Glazed’s command and field abstractions because they happen to be mounted through Cobra. Glazed already has typed field definitions, reusable sections, source middleware, semantic decoding, provenance, and a frontend-neutral runner. Replacing these components would erase the exact architecture needed to build a better native CLI.

The problem is therefore one of **architectural separation**. The migration must discover which responsibilities are semantic and which are frontend-specific.

Definition 1.1 (semantic responsibility). A responsibility is semantic when it describes what an application input *means* independently of how a user supplies it. Examples include field identity,

integer versus string type, legal choices, requiredness after source resolution, and the command effect that consumes the result.

Definition 1.2 (frontend responsibility). A responsibility is frontend-specific when it describes how one external interface encodes or transports semantic inputs. Examples include `--limit`, `-n5`, environment variable naming, JSON property names, shell-completion framing, and terminal layout.

The distinction is applied immediately:

Concern	Semantic or frontend?	Reason
field <code>search.limit</code> has type integer	semantic	every frontend must produce an integer for the same slot
CLI spelling <code>--limit</code>	frontend	another frontend may use <code>limit</code> in JSON or <code>GOLEM_LIMIT</code> in the environment
<code>limit</code> defaults to 20	semantic policy	the default belongs to value resolution, not token spelling
<code>-n5</code> is accepted	CLI grammar	this is a lexical convention
<code>RunIntoGlazeProcessor</code> receives resolved values	semantic execution	the handler should not care which frontend supplied them
terminal help groups output fields	frontend presentation over semantic metadata	grouping is a view of the model

Counterexample - the field name is not the flag name. If a field is named `api_token`, normalized to CLI spelling `--api-token`, and placed in a prefixed section that exposes `--auth-api-token`, then checking requiredness under the observed CLI key is wrong. Requiredness belongs to the semantic field identity, not whichever spelling was used to reach it.

1.3 Research questions

The study is organized around five research questions.

RQ1 - Semantic preservation. Which Glazed abstractions are already independent of Cobra, and what laws must remain true after the dependency is removed?

RQ2 - Native language design. What command and option grammar should Glazed own, and how can it be expressed as a deterministic parser with structured diagnostics?

RQ3 - Multi-source resolution. How should CLI occurrences participate in the existing defaults/configuration/environment precedence model without making CLI parsing a special source of truth?

RQ4 - Multiple interpreters. How can parsing, completion, short help, long help, machine manifests, and execution share one compiled model while retaining different outputs and failure policies?

RQ5 - Migration evidence. What experimental and differential evidence is sufficient to replace Cobra without silently changing downstream applications?

The thesis hypothesis is:

Hypothesis. Glazed can remove Cobra most safely by compiling its existing command descriptions, sections, fields, aliases, and contracts into an immutable command catalog, then implementing parsing, completion, help, manifests, and a transitional Cobra adapter as interpreters over that catalog.

This hypothesis is stronger than “a custom parser can scan flags.” It predicts a package structure, a migration order, and a set of testable equivalence laws.

1.4 The current architecture

Figure 1.1 summarizes the pinned architecture.

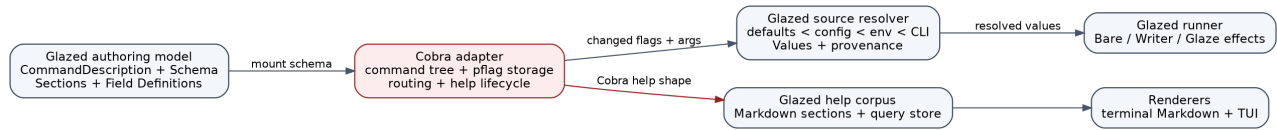


Figure 1.1: Current Glazed architecture: Glazed owns the semantic model and source resolver, while Cobra supplies the outer command and flag machinery.

The left side is Glazed-native. `CommandDescription` owns command identity and prose. `Schema` owns ordered sections. Each section owns `Definition` values describing names, types, defaults, choices, requiredness, and whether a field is positional. `Definition.ParseField` converts one or more raw strings into a semantic value. `sources.Execute` clones a schema and runs an ordered middleware chain. `values.FieldValue` stores both the winning value and a log of parse steps. `runner.RunCommand` dispatches to `BareCommand`, `WriterCommand`, or `GlazeCommand` without requiring Cobra.

The red boundary is the coupling. `pkg/cli/cobra.go` builds `*cobra.Command` values. `CobraParser` arranges `FromCobra` and `FromArgs` source middleware. `schema.CobraSection` and `fields/cobra.go` teach sections and definitions how to mount and read pflag values. `pkg/help/cmd/cobra.go` receives `*cobra.Command`, calls Cobra methods to derive paths, flags, aliases, groups, and usage, and injects those values into help templates.

The architecture has an important asymmetry: Glazed’s semantic model is richer than Cobra’s flag model, but the help adapter often reconstructs Glazed concepts through Cobra objects. A native design should reverse that direction. Cobra, if retained temporarily, should be a view of the compiled Glazed catalog; Glazed help should never need a Cobra command to understand a Glazed field.

1.5 Command descriptions as language declarations

Motivation. A parser cannot be the authority for a command language if the same commands must also be loaded from YAML, exposed through HTTP, described to agents, or executed in tests. The reusable object is the declaration of the operation, not the parser node.

Definition 1.3 (command declaration). A command declaration is a tuple

$$C = (p, d, S, k, e, m),$$

where:

- $p \in \Sigma_C^*$ is the command path;
- d is authored prose such as short and long descriptions;
- S is an ordered input schema;
- k is the command kind, such as bare, writer, or structured-row command;
- e is the effect implementation;
- m is additional metadata and source provenance.

Glazed's `CommandDescription` already supplies most of this tuple. The command interfaces supply k and e . A future static command contract can make output and effect declarations explicit rather than inferred from Go interfaces.

A minimal target API could preserve the existing authoring style:

```
type Command interface {
    Description() *CommandDescription
}

type BareCommand interface {
    Command
    Run(context.Context, *values.Values) error
}

type WriterCommand interface {
    Command
    RunIntoWriter(context.Context, *values.Values, io.Writer) error
}

type GlazeCommand interface {
    Command
    RunIntoGlazeProcessor(
        context.Context,
        *values.Values,
        middlewares.Processor,
    ) error
}
```

The key migration constraint is negative: no method in these interfaces should mention Cobra, pflag, or a terminal renderer.

1.5.1 Worked example: declaring note search

The semantic command is:

```
cmds.NewCommandDescription(
    "search",
    cmds.WithParents("note"),
    cmds.WithShort("Search notes"),
    cmds.WithFlags(
        fields.New("limit", fields.TypeInteger,
            fields.WithDefault(20)),
        fields.New("tag", fields.TypeStringList),
    ),
    cmds.WithArguments(
        fields.New("query", fields.TypeString,
            fields.WithRequired(true)),
    ),
)
```

Nothing in the meaning of this command requires `--limit`. The semantic slots are `limit`, `tag`, and `query`. A CLI binding may expose `--limit`, an environment binding may expose `GOLEM_NOTE_SEARCH_LIMIT`, and a JSON binding may expose `{ "limit": 20 }`.

1.5.2 Worked example: the same command through a non-CLI frontend

A local RPC adapter could receive:

```
{
  "command": ["note", "search"],
  "inputs": {
    "query": "snapshot cuts",
    "limit": 5,
    "tag": ["architecture", "streaming"]
  }
}
```

It can populate `values.Values` and call `runner.RunCommand` without constructing a Cobra command. This is not hypothetical architecture: the current runner already accepts resolved values directly.

1.6 Sections and field definitions form a typed signature

Motivation. A large CLI needs reusable groups such as logging, database, output, and credentials. Flat flag maps obscure provenance and collide when two concerns use the same word. Glazed sections preserve grouping and ordering.

Definition 1.4 (field signature). For a command C , its semantic field signature is a finite family

$$\mathcal{F}_C = \{f_1 : \tau_1, \dots, f_n : \tau_n\},$$

where each f_i has a stable semantic identity and each τ_i is a field type such as string, integer, Boolean, choice, list, file-derived value, key-value map, or object.

A field also carries constraints and policies:

$$f = (\text{id}, \tau, \text{required}, \text{default}, \text{choices}, \text{position}, \text{metadata}).$$

Definition 1.5 (sectioned schema). A sectioned schema is an ordered sequence

$$S = \langle (s_1, \mathcal{F}_1), \dots, (s_k, \mathcal{F}_k) \rangle.$$

The order matters for authored help and deterministic manifests even when lookup is map-like.

Fundamentals - a signature is not a value. In type theory and algebra, a signature declares the names and shapes of operations or data. A Glazed schema declares which fields may exist and how to interpret them. `values.Values` is an assignment under that signature. Confusing the two leads to mutable parsers that treat “defined,” “provided,” and “resolved” as the same state.

1.6.1 Worked example: two sections, one semantic namespace

Suppose `note search` has:

section default:

```
query      string, positional, required
limit      integer, default 20
tag        string-list
```

section output:

```
format     choice(table,json,jsonl,csv,tsv), default table
max-rows   integer, optional
```

A help renderer may show two groups. A config file may store them as nested mappings. A CLI may prefix output fields, or may choose not to. The semantic IDs should remain stable:

```
default.query
default.limit
default.tag
output.format
output.max-rows
```

The existing code often uses section slugs and field names in this role. The proposed compiler makes the identity explicit and validates that it is unique in the effective command scope.

1.7 Semantic codecs: the parser should not own types

Motivation. Token scanning answers questions such as “did `--limit` receive the next token?” It should not duplicate all conversions for integers, choices, lists, files, dates, secrets, and object values. Glazed already owns those conversions in `Definition.ParseField`.

Definition 1.6 (raw occurrence). A raw occurrence is a record

$$o = (f, r, i, \phi),$$

where f is a semantic field identity, r is raw text, i is the token position, and ϕ records the syntactic form such as `long-equals`, `short-attached`, or `operand`.

Definition 1.7 (semantic codec). A semantic codec for type τ is a partial function

$$\text{decode}_\tau : \text{Raw}^* \rightharpoonup V_\tau.$$

It is partial because text may be malformed or violate choices. The scanner produces occurrences; the field codec produces typed values.

1.7.1 Worked example: three surface forms, one codec input

These invocations are lexically different:

```
--limit=5
--limit 5
-n5
```

After scanning they all produce a raw occurrence for semantic field `default.limit` with raw text “5”. The integer codec then produces value 5. This normalization creates a powerful invariant:

$$\text{decode}(\text{scan}(a)) = \text{decode}(\text{scan}(b))$$

for surface forms a and b that are intended to be aliases.

1.7.2 Counterexample: parser-specific typed storage

If pflag stores an integer directly and an HTTP frontend stores JSON numbers directly while configuration files store strings, each frontend quietly develops its own coercion rules. A value accepted from the CLI may be rejected from a config file, or vice versa. The safer design has frontend decoders produce either a common raw representation or an explicitly typed intermediate consumed by one field validator.

1.8 Source middleware and provenance are already a resolution algebra

Motivation. A required field may come from a configuration file or environment variable. Validating required CLI flags before other sources are considered is therefore incorrect. Glazed fixed precisely this class of bug by deferring required validation until after all sources merge.

The current source chain is one of Glazed’s strongest architectural assets. Middleware can load defaults, several config files, environment variables, provided values, CLI occurrences, or custom sources. `FieldValue` retains a log of parse steps, so the final assignment is explainable.

Definition 1.8 (source assignment). A source assignment is a finite partial map

$$M_s : \text{FieldID} \rightarrow \text{Raw}^*.$$

The source name s may be `default`, `config:base.yaml`, `config:local.yaml`, `env`, `profile`, or `cli`.

Definition 1.9 (resolved value with provenance). A resolved value is a pair

$$(v, \ell),$$

where v is the winning typed value and ℓ is an ordered trace of all source contributions that were considered or merged.

This design is richer than the ordinary flag model. A native parser should become one producer of M_{cli} ; it should not replace the resolution algebra.

1.8.1 Worked example: explain a winning value

Assume:

```
default limit = 20
base config limit = 100
local config limit = 50
environment limit = 25
CLI --limit 5
```

The final value is 5, but an explain command can show:

```
default      20
config:base  100
config:local  50
env          25
cli          5   <- winner
```

A parser that writes only `map[string]any{"limit": 5}` destroys this evidence.

1.9 The frontend-neutral runner closes the semantic loop

`pkg/cmds/runner.RunCommand` accepts a context, a Glazed command, resolved values, and optional output dependencies. It selects the command interface and invokes the appropriate effect. For structured commands, it constructs or accepts a processor and closes it after execution.

Definition 1.10 (execution plan). An execution plan is a tuple

$$P = (C, V, O, \Gamma),$$

where C is a compiled command, V is a validated resolved assignment, O is output configuration, and Γ is the execution context and dependency environment.

The runner interprets P as effects. It should not decide how command tokens were scanned. This boundary implies a migration strategy: build the native compiler and parser until they can produce the same V accepted by the current runner.

1.10 Aliases are macros, not merely alternate names

Glazed aliases can name a target command and prepopulate flags and arguments. This is more expressive than a Cobra command alias, which usually names the same node. The difference matters.

Definition 1.11 (name alias). A name alias maps one command token to the same command node without changing the remaining invocation.

Definition 1.12 (command macro alias). A command macro alias is a rewrite rule

$$a\ x \Longrightarrow p\ q\ x,$$

where a is the alias path, p is the target command path, q is a fixed token prefix, and x is the caller-supplied remainder.

For `golem`:

```
find QUERY --tag architecture
```

rewrites to

```
note search --format=jsonl QUERY --tag architecture
```

A compiler must validate alias target existence, path collisions, and cycles. Runtime alias expansion should record both the invocation path and the canonical command path so diagnostics and provenance can explain what happened.

1.10.1 Counterexample: copying the target schema at alias construction

If an alias copies a mutable target schema and the target later gains a field, the alias may silently drift. The current Glazed alias description clones the target schema at runtime, which protects some mutation boundaries but also shows that alias semantics are phase-sensitive. A compiled catalog can resolve aliases after all commands are loaded and then freeze the effective target schema once.

1.11 The help system is modern in content, legacy in its command adapter

Glazed already has more than conventional `--help` output. Help sections are authored in Markdown with typed section categories, topics, related commands, related flags, display metadata, package identity, and version. The help system can query these sections, render Markdown through Glamour, and browse them in a Bubble Tea interface with search, lists, a viewport, a cheatsheet, and clipboard support.

The modernization opportunity is therefore not “replace plain text with color.” It is to join two models that are currently adjacent:

1. the exact command and field schema;
2. the authored conceptual help corpus.

The current Cobra adapter reconstructs command facts through `*cobra.Command`: paths, aliases, available subcommands, local and inherited flags, use lines, and padding. A native catalog can supply those facts directly and with stronger semantic identities. The existing corpus and TUI can remain.

Definition 1.13 (reference help). Reference help is exact, model-derived information about command paths, inputs, defaults, choices, aliases, examples, and output contracts.

Definition 1.14 (conceptual help). Conceptual help is authored material that explains why, when, workflows, examples, tutorials, caveats, and relationships.

A modern help system combines the two without pretending that one can replace the other.

1.12 Historical repairs as design evidence

Recent Glazed history contains several repairs that directly inform the replacement.

- Required-field validation was moved until after all configured sources were parsed. This confirms that requiredness is a property of the final resolved assignment, not presence on the CLI.
- Cobra builders were changed to use RunE so command errors propagate to Execute instead of being terminated or swallowed in a non-returning callback. This confirms that error transport is part of the frontend contract.
- The broad structured-output option surface was reduced to a smaller serialization boundary. This confirms that command compilation should distinguish application inputs from framework-injected inputs and validate collisions atomically.
- A linter was added to discourage applications from bypassing Glazed definitions with raw Cobra, pflag, or standard-library flag declarations. This confirms that Glazed intends its schema to be the semantic authority even while Cobra remains the implementation frontend.

These repairs support a general rule:

Research finding. Compile and interpret Glazed’s semantic model first. Treat process adapters as replaceable projections. Bugs recur when a frontend is allowed to validate, terminate, or enlarge the command contract independently of the Glazed model.

1.13 Why the existing string-list parser is a seed, not the final design

`fields.GatherFlagsFromStringList` demonstrates that Glazed can parse raw token slices without Cobra. It supports long and short names, `--name=value`, booleans, repeated lists, defaults, and the existing semantic codecs. It is valuable evidence and should be mined for compatibility tests.

It is not yet a framework parser. The source itself records a TODO for the `--` terminator. It does not select commands, compute inherited scopes, expose partial states for completion, return structured diagnostics, or validate the whole command set. More subtly, it keys raw occurrences by the spelling observed on the command line but checks requiredness by `Definition.Name`. A required field supplied through a short flag, a prefixed flag, or an underscore-to-hyphen normalized flag can therefore be absent from the requiredness lookup even though it was parsed.

The mechanized source audit included with this thesis yields:

Semantic name	Observed key	Required check finds it?	Form
required	required	yes	canonical long spelling
required	r	no	short spelling
api_token	api-token	no	normalized spelling

Semantic name	Observed key	Required check finds it?	Form
path	config-path	no	prefixed spelling

This counterexample motivates the binding separation developed in Chapter 2.

1.14 Chapter synthesis

Glazed is not starting from zero. Its current architecture already separates semantic decoding, multi-source resolution, provenance, and effect execution from the CLI in important places. The Cobra dependency persists because command compilation, token grammar, completion, and parts of help are not yet owned by a Glazed-native catalog.

Figure 1.2 reframes the system as a latent language runtime.

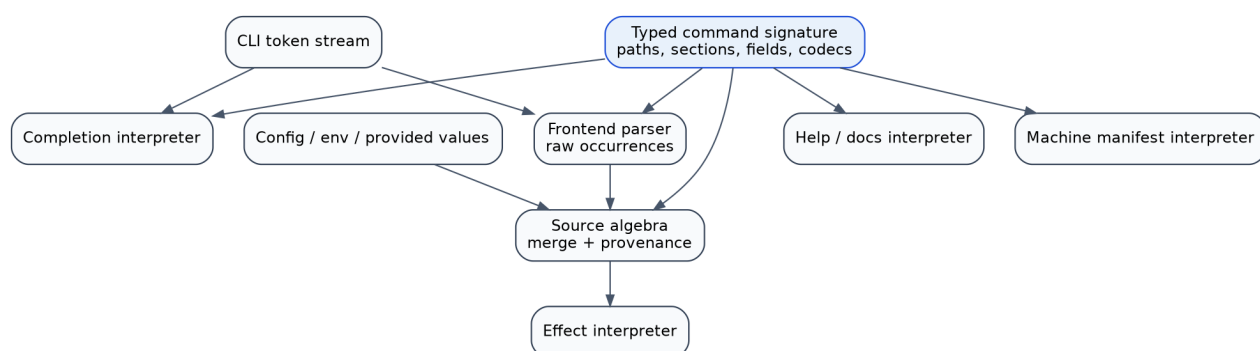


Figure 1.2: Glazed’s latent language runtime: one typed model can feed parsing, source resolution, effects, completion, help, and manifests.

The target is not to make every package know about a new parser. The target is to create one compiled model that makes the parser one interpreter among several.

1.15 Exercises for Chapter 1

1. Classify each of the following as semantic, frontend-specific, or mixed: a choice list, a short flag, a default value, a terminal column width, an environment prefix, a file-loading field type, and a “dangerous effect” warning.
2. Draw the dataflow for a required field satisfied by a configuration file while `--help` is requested. At which stage must required validation be bypassed or deferred?
3. Give two reasons that `CommandDescription` should not itself store a mutable parser object.
4. Construct a command with two sections whose fields share the display name format. Propose stable semantic IDs and distinct CLI bindings.
5. Explain why `Definition.ParseField` should survive a Cobra removal even if its API eventually changes.
6. Model a Glazed alias that fixes `--format=jsonl` and `--limit=10`. Is it a name alias or a macro alias? What happens when the caller also supplies `--limit=20` under first-wins, last-wins, and reject-repeat policies?
7. Read the current `runner.RunCommand` API. List the minimum additional information required to turn a validated value assignment into an executable plan.
8. The help corpus links sections to command and flag strings. Explain why stable semantic IDs would be safer than strings after aliases, prefixes, and deprecations are introduced.

9. Design a test that demonstrates the required-short-flag key mismatch in the existing string-list parser without depending on Cobra.
10. State one law that must be preserved exactly during migration and one behavior that should intentionally change in a major release.

Chapter 2

A Mathematical Semantics for Typed Command Lines

2.1 Learning objectives

After this chapter, the reader should be able to:

- model a command set as a rooted trie and its accepted invocations as a language;
- distinguish semantic field identity from CLI bindings and prove why binding injectivity is required;
- specify a deterministic scanner as a transition system over token sequences;
- define positional grammars, repeat policies, source precedence, and provenance precisely;
- model aliases as terminating rewrites and completion as a residual language;
- design diagnostics as structured semantic results rather than formatted strings.

2.2 Motivation: compatibility requires a language specification

Cobra removal is sometimes discussed as though the target behavior were obvious. It is not. Consider only six questions:

Does `-abc` mean three Boolean options, one option named `abc`, or `-a` with value `bc`?

Does `--no-color` negate a Boolean field, or is it an unrelated long name?

After the first positional argument, may later options still appear?

Does `--` stop only option parsing, or command descent as well?

When a list is set in config and repeated on the CLI, is it appended or replaced?

If an alias injects `--limit=10` and the caller supplies `--limit=20`, which wins?

Different CLI libraries answer these questions differently. Even within one library, configuration flags can change the answer. A replacement cannot be tested against “normal CLI behavior.” It needs an explicit language definition.

A language definition has three levels:

1. **static language** - commands, fields, bindings, aliases, and constraints that exist after compilation;
2. **lexical and syntactic language** - token sequences accepted by the CLI frontend;
3. **semantic language** - resolved typed assignments after all sources merge.

The native design should make transitions between these levels explicit.

2.3 Command paths form a prefix language

Let Σ_C be the finite vocabulary of command-name tokens.

Definition 2.1 (command path language). A command path language is a finite set

$$L_C \subseteq \Sigma_C^*$$

that is represented by a rooted trie. A word $p \in L_C$ denotes a declared command. A proper prefix of p may denote either another declared command or a structural namespace.

For `golem`:

$$L_C = \{\langle \text{note}, \text{search} \rangle, \langle \text{note}, \text{show} \rangle, \langle \text{cache}, \text{clear} \rangle\}.$$

The trie has root children `note` and `cache`. The intermediate node `note` may exist only to organize children; it need not be runnable.

Definition 2.2 (structural namespace). A structural namespace is a trie node with children but no effect implementation. It is legal as a command prefix and illegal as a final executable selection unless the frontend chooses to render help instead.

This definition eliminates an accidental dependency on Cobra’s ability to create placeholder parent commands. The compiled catalog can represent namespaces directly.

2.3.1 Worked example: path resolution

Given tokens

`note search "typed CLI" --limit 5`

command resolution consumes the longest initial command word accepted by the trie:

$$\text{resolvePath}(w) = (\langle \text{note}, \text{search} \rangle, \langle \text{typed CLI}, \text{--limit}, 5 \rangle).$$

This is not necessarily a simple longest-prefix scan because options may be interspersed and inherited options may occur before a subcommand. The parser therefore carries both the current trie node and the effective option scope.

2.3.2 Counterexample: guessing namespaces at runtime

If the runtime creates a namespace whenever it sees an unknown path segment, then a typo such as `golem ntoe search` can create a plausible but empty path rather than a diagnostic. Namespace creation belongs to compilation, where the complete command set is known.

2.4 Semantic fields and frontend bindings

The central formal separation is between a field and the strings used to address it.

Definition 2.3 (semantic field). A semantic field is a tuple

$$f = (\iota, \tau, R, D, C, M),$$

where:

- $\iota \in I$ is a stable field identity;

- τ is its semantic type;
- R is a requiredness predicate;
- D is an optional default;
- C is a set of value constraints such as choices;
- M is descriptive metadata.

Definition 2.4 (CLI binding). A CLI binding for field f is a tuple

$$b_{cli}(f) = (N_l, N_s, a, o, s, h, d),$$

where:

- N_l is a finite set of long names;
- N_s is a finite set of short runes;
- a is argument arity;
- o is occurrence policy;
- s is splitting policy for list-like text;
- h states visibility in help;
- d stores deprecation and replacement metadata.

A field may have zero or more bindings for other frontends:

```

type FieldSpec struct {
    ID      FieldID
    Type     Type
    Required RequiredPolicy
    Default  *RawValue
    Choices  []RawValue
}

type CLIFieldBinding struct {
    FieldID      FieldID
    Long          []string
    Short         []rune
    Arity         Arity
    Repeat        RepeatPolicy
    Split         SplitPolicy
    Scope         ScopePolicy
    Hidden        bool
    Deprecated    *Deprecation
    NegatedLong   string
}

type EnvFieldBinding struct {
    FieldID FieldID
    Names   []string
}

type ConfigFieldBinding struct {
    FieldID FieldID
    Paths   [][]string
}

```

Definition 2.5 (effective binding relation). For command node v , let

$$B_v : \text{CLIName} \rightarrow I$$

map every visible long or short name to one semantic field identity.

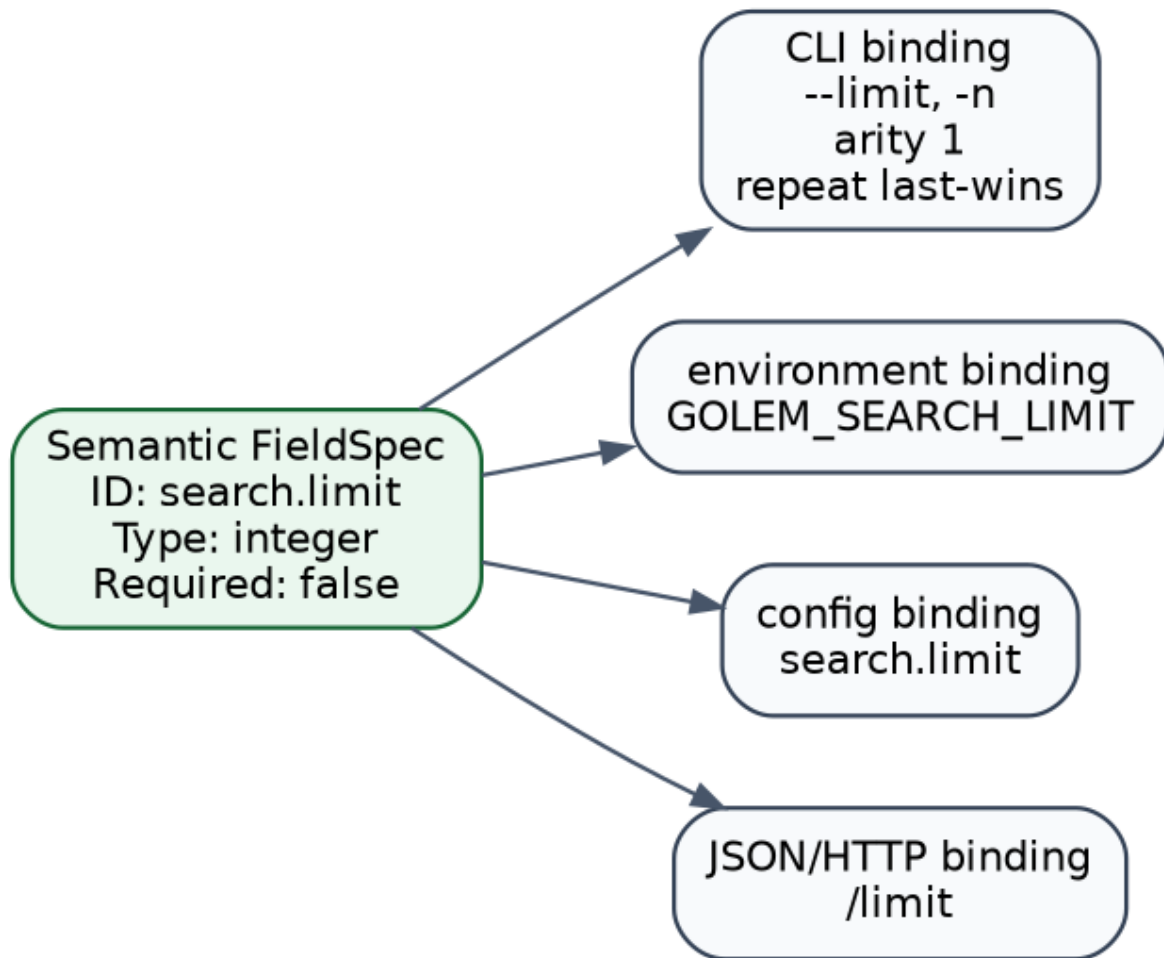


Figure 2.1: One semantic field can have several frontend bindings without changing identity.

Compile-time law 2.1 (binding injectivity). Within an effective command scope, every CLI name denotes at most one semantic field:

$$B_v(n) = i \wedge B_v(n) = j \implies i = j.$$

This law should be checked for every command after inheritance and framework injection, not only inside each authored section.

2.4.1 Worked example: prefixed sections

Suppose the logging section contains semantic field `logging.level`. One application exposes it as `--log-level`; another embeds the same section twice:

```
--client-log-level
--server-log-level
```

The semantic model should not mutate the original field name into these strings. Compilation creates two instantiated field identities or two scoped bindings:

```
client.logging.level -> --client-log-level
server.logging.level -> --server-log-level
```

Requiredness, configuration paths, and help links refer to the semantic identities. Token scanning refers to the bindings.

2.4.2 Worked example: deprecated spellings

A field may accept both `--output` and `--format` during a migration:

```
CLIFieldBinding{
  FieldID: "output.format",
  Long:    []string{"format", "output"},
  Deprecated: &Deprecation{
    Name:      "output",
    Replacement: "format",
  },
}
```

Both names produce occurrences for the same identity. The diagnostic layer can warn on the deprecated form without duplicating the semantic field.

2.4.3 Counterexample: string keys as semantic identity

The existing string-list parser builds a lookup map containing long, short, normalized, and prefixed spellings, then stores raw values under the spelling that was observed. It later checks requiredness by `Definition.Name`. The condition

$$\text{observed spelling} = \text{semantic name}$$

is false for short, normalized, and prefixed forms. The bug disappears when every binding maps immediately to one ι .

2.5 Whole-program compilation

Motivation. Many invalid states cannot be detected while parsing one invocation. A short flag can collide only after inherited sections are combined. An alias target can be validated only after

all commands are loaded. A help link can be checked only when the catalog is complete. These are compiler problems.

Definition 2.6 (authored program). An authored command program P is a finite collection of mutable command declarations, section instances, field specifications, frontend bindings, aliases, contracts, and authored help references.

Definition 2.7 (compiled command catalog). A compiled catalog is an immutable tuple

$$K = (T, \{\mathcal{F}_v\}, \{B_v\}, A, H, Q),$$

where:

- T is the normalized command trie;
- $\mathcal{F}_v$ is the effective semantic field signature at node v ;
- B_v is the effective CLI binding relation;
- A is the validated alias rewrite system;
- H is indexed help and example metadata;
- Q is a set of precomputed lookup structures and provenance records.

Compilation is a partial function:

$$\text{compile} : P \rightharpoonup K.$$

It returns either a complete catalog or a deterministic set of diagnostics. It must not partially mutate a live command tree.

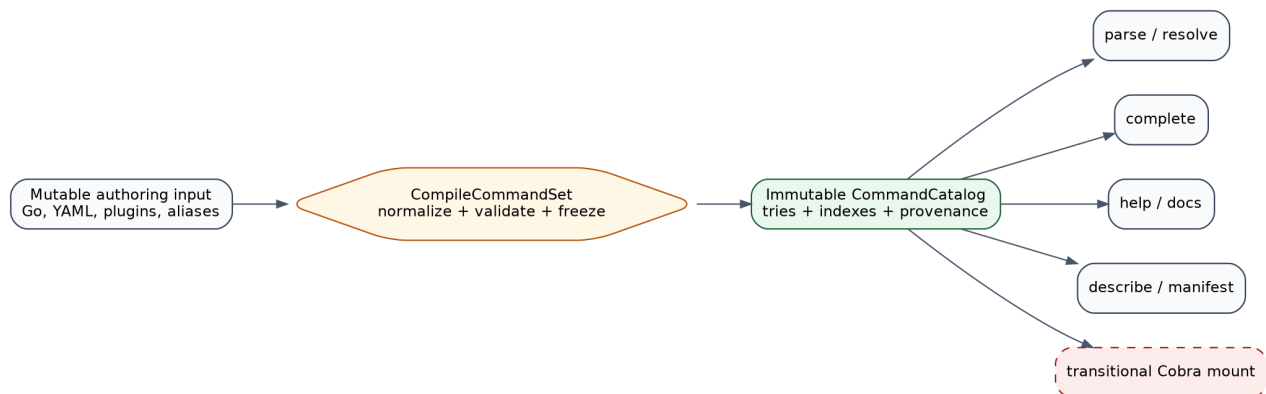


Figure 2.2: The proposed compile/runtime split. Mutable definitions are normalized, validated, and frozen before any frontend interprets them.

2.5.1 Compile-time obligations

At minimum, compilation validates:

1. command paths are non-empty and normalized;
2. sibling primary names and aliases do not collide;
3. semantic field identities are unique in each effective scope;
4. long and short bindings are injective in each effective scope;
5. positional grammars are well formed;
6. choice defaults are legal;
7. aliases target existing commands and form a terminating rewrite system;
8. help references point to known command and field identities;

9. framework-injected fields do not collide with application fields;
10. secret defaults are not exported in public manifests.

2.5.2 Worked example: atomic collision reporting

Suppose a command declares application field `--format`, while an output section also injects `--format`. A mount-as-you-go builder may add earlier commands, fail at this command, and leave later commands absent. A compiler instead reports:

```
{
  "code": "E_FLAG_COLLISION",
  "path": ["note", "search"],
  "field": "output.format",
  "message": "--format collides with application.format"
}
```

No catalog is returned. The application cannot start with a partial command set.

2.6 Positional arguments form a small grammar

Glazed currently preserves positional order and permits a list-valued final argument to consume the remainder. The native compiler should state that law explicitly.

Let a positional declaration be one of:

- $\text{Req}(f)$ - exactly one required operand;
- $\text{Opt}(f)$ - zero or one operand;
- $\text{Rest}(f)$ - zero or more operands assigned to a list field.

Definition 2.8 (well-formed positional grammar). A positional sequence is well formed when it belongs to

$$\text{Req}^* \text{Opt}^* (\text{Rest})?,$$

meaning required operands precede optional operands, and a variadic operand may occur only at the end.

This constraint makes binding deterministic without backtracking.

2.6.1 Worked example 2.5

Valid:

```
QUERY
QUERY [SCOPE]
QUERY [SCOPE] [FILE...]
```

Invalid:

```
[QUERY] REQUIRED_SCOPE
FILE... OUTPUT
```

The first invalid grammar cannot decide whether one operand belongs to the optional first slot or the required second slot without a policy. The second leaves no input for the field after the variadic field.

2.6.2 Counterexample: positional parsing before command selection

If positional fields are bound before the final command is known, the token search may be consumed as a root operand rather than a subcommand. Command selection and option scope must be part of the same state machine.

2.7 A deterministic scanning transition system

Motivation. Informal token loops become fragile around clustered short flags, missing values, --, command descent, and interspersed options. A transition system makes every decision state-dependent and testable.

Let the input be $w = t_0 t_1 \dots t_{n-1}$.

Definition 2.9 (scanner state). A scanner state is

$$q = (v, i, m, E, O, R, \ell),$$

where:

- v is the current command trie node;
- i is the next token index;
- $m \in \{\text{Command}, \text{Expect}(f), \text{Operand}\}$ is the mode;
- E indicates whether -- has ended option recognition;
- O is the multimap of raw field occurrences;
- R is the ordered operand list;
- ℓ is an optional trace of transitions.

The transition relation

$$q \xrightarrow{t_i} q'$$

is deterministic for a compiled catalog because command edges and option bindings are injective.

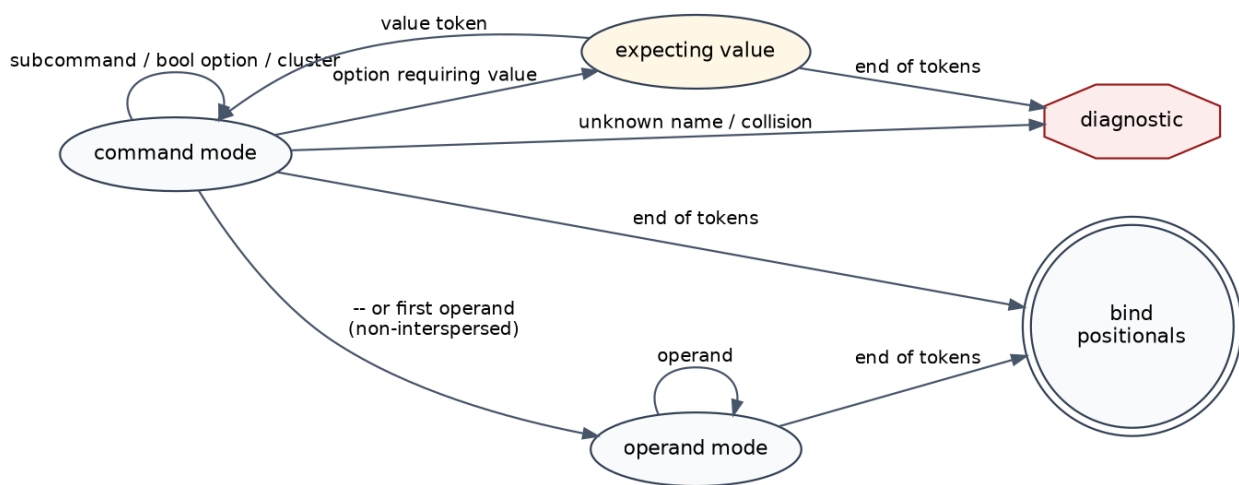


Figure 2.3: A simplified parser state machine.

2.7.1 Core transition rules

The rules below are schematic.

Long Boolean option. If $t_i = --n$ and $B_v(n) = f$ with Boolean type:

$$(v, i, \text{Command}, 0, O, R, \ell) \rightarrow (v, i + 1, \text{Command}, 0, O[f \mapsto O(f) \cdot \text{true}], R, \ell').$$

Long valued option. If $t_i = --n$ and field f has arity one, the next token or the text after $=$ becomes one raw occurrence.

Negated Boolean. If `no-` is enabled for f , `--no-name` adds raw `false`. It is not a universal transformation: a literal field named `no-cache` must remain addressable.

Short cluster. For token `-vn5`, consume runes from left to right. Boolean v binds `true`. Non-Boolean n consumes the remaining text 5 as its value and terminates the cluster.

Terminator. Token `--` sets $E = 1$. Every later token is an operand even when it begins with `-` or matches a command name.

Command edge. In command mode, if a token names a child edge and the interspersed policy permits descent, update v and continue.

Operand. Otherwise append the token to R . Under non-interspersed policy, enter operand mode permanently.

2.7.2 Worked example: tracing an alias and a cluster

Parse:

```
golem find -vn5 --tag=a,b -- --literal
```

The alias rewrites `find` to target path `note search` with prefix `--format=jsonl`. The canonical token stream becomes conceptually:

```
note search --format=jsonl -vn5 --tag=a,b -- --literal
```

A trace is:

Token/form	State	Action
find	root	expand alias; select note search
injected --format=jsonl	note search	bind output.format to jsonl
-v	note search	bind global verbose=true
-n5	note search	bind limit=5
--tag=a,b	note search	bind two list texts after split policy
--	note search	enter operand-only mode
--literal	operand mode	bind required positional query

The prototype records both canonical `CommandPath` and user-facing `InvocationPath`, allowing help and errors to choose the appropriate identity.

2.7.3 Worked example: negative numbers

Consider:

```
calculate --offset -5
```

The parser is in `Expect(offset)` after `--offset`, so `-5` is a value even though it begins with `-`. A token classifier that simply treats every leading hyphen as an option will reject valid numeric input.

2.7.4 Counterexample: Boolean options that optionally consume a value

Some libraries permit `--verbose false`, while others interpret `false` as an operand because bare `--verbose` already completes the Boolean option. Supporting both without ambiguity requires a declared arity policy. The native language should choose one of:

- implicit true, explicit false only through `--verbose=false` or `--no-verbose`;
- mandatory Boolean value;
- lookahead with a closed Boolean vocabulary.

The prototype uses the first policy because it is deterministic and familiar.

2.8 Raw occurrences precede repeat policy

A field may occur more than once:

```
--tag architecture --tag cli
--limit 10 --limit 20
```

Definition 2.10 (occurrence multimap). The scanner result is

$$O : I \rightarrow \text{Occurrence}^*,$$

where ordering is token order.

Definition 2.11 (repeat policy). A scalar field's repeat policy is one of:

- reject - more than one occurrence is an error;
- first-wins - decode the first occurrence;
- last-wins - decode the last occurrence.

List fields usually concatenate occurrences *within the same source*, but this should be declared separately from cross-source merge.

2.8.1 Worked example: why two policies are needed

For a list field `tag`:

```
config: [research]
env:    [production]
CLI:    --tag architecture --tag cli
```

Within the CLI source, repeated occurrences naturally yield `[architecture, cli]`. Across sources, two reasonable policies exist:

```
replace -> [architecture, cli]
append  -> [research, production, architecture, cli]
```

List typing alone does not determine which is correct. The prototype uses replacement as the compatibility default and explicit `MergeAppend` for opt-in accumulation.

2.9 Source precedence as ordered partial-map override

Let source maps be D (defaults), $F_1, \dots, F_k$ (config overlays), E (environment), P (programmatically provided values), and C (CLI).

Definition 2.12 (right-biased override). For partial maps $A, B : I \rightarrow V$:

$$(A \triangleright B)(i) = \begin{cases} B(i), & i \in \text{dom}(B), \\ A(i), & \text{otherwise.} \end{cases}$$

The ordinary replacement resolution is:

$$R = D \triangleright F_1 \triangleright \dots \triangleright F_k \triangleright E \triangleright P \triangleright C.$$

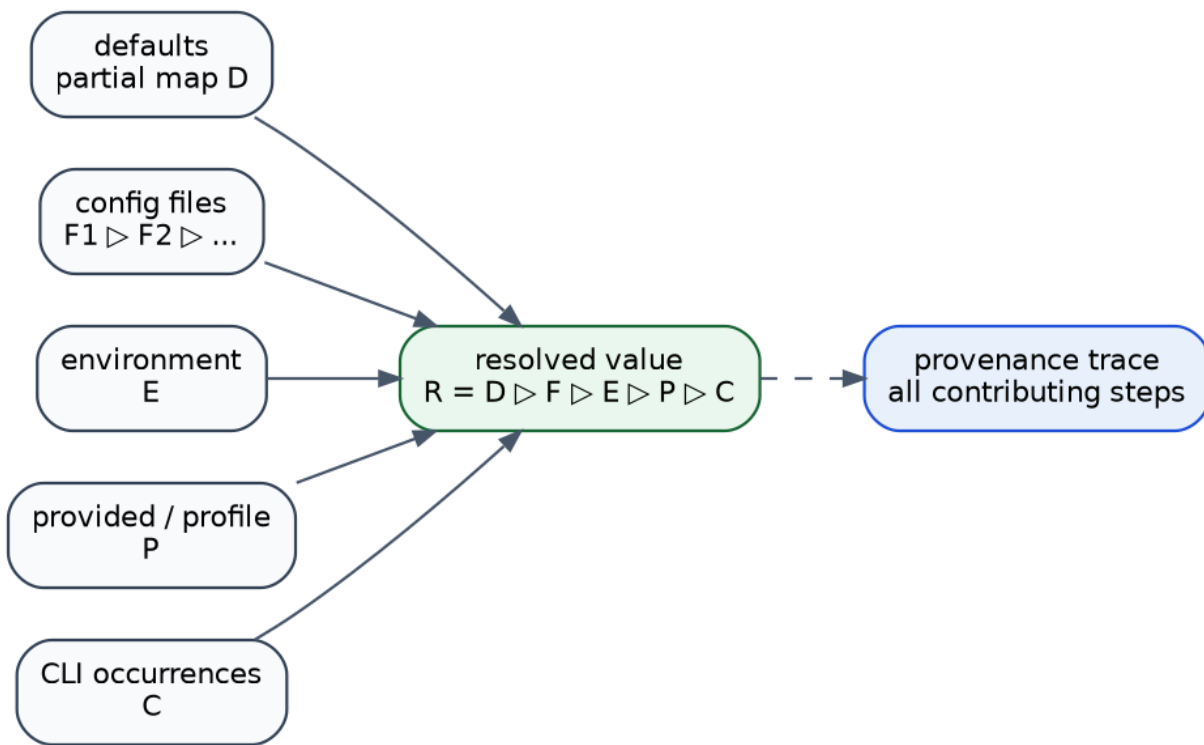


Figure 2.4: Source resolution as an ordered fold over partial maps, preserving a provenance trace.

Proposition 2.1. Right-biased override is associative:

$$(A \triangleright B) \triangleright C = A \triangleright (B \triangleright C).$$

Proof sketch. For each field identity i , inspect whether i is defined in C , then B , then A . Both groupings select the rightmost defined source. Since partial maps are equal pointwise, the maps are equal. $\square$

Associativity allows source middleware to be grouped without changing replacement semantics, provided the source order is preserved.

2.9.1 Provenance-enriched fold

A winning value alone is insufficient. Define each field result as

$$R(i) = (v_i, \ell_i),$$

where ℓ_i records source, raw form, parsed value, and relevant metadata. The merge operator updates v_i according to policy and appends a step to ℓ_i .

2.9.2 Worked example: requiredness after merge

Let endpoint be required. The CLI does not provide it, but the environment map does:

$$E(\text{endpoint}) = \text{https://api.example.}$$

After folding, endpoint is in $\text{dom}(R)$, so required validation succeeds.

Definition 2.13 (post-merge requiredness). The requiredness predicate is:

$$\text{ValidRequired}(R) \iff \forall f \in \mathcal{F}_v, \text{required}(f) \Rightarrow \iota_f \in \text{dom}(R).$$

It is intentionally evaluated after all configured sources and alias defaults have been interpreted.

2.9.3 Counterexample: pflag-level required markers

A pflag-level marker can require that a flag was changed on the command line. That is a different predicate:

$$\text{requiredCLI}(f) \Rightarrow \iota_f \in \text{dom}(C).$$

Glazed normally wants required semantic values, not required CLI occurrences. The native model should support both only if they are named distinctly.

2.10 Defaults, absence, null, and empty values

A typed source system needs at least four states:

1. field not supplied;
2. field supplied with an empty but valid value, such as "" or [];
3. field supplied with explicit null, if the type permits it;
4. field supplied with a non-empty value.

Using `nil` for all four makes defaults and requiredness unreliable.

Definition 2.14 (presence). Presence is a semantic property separate from the value. A source contribution is an option-like value

$$\text{Present}(v) \text{ or } \text{Absent}.$$

If null is supported, it is a value inside `Present` rather than an alias for absence.

2.10.1 Worked example: empty list overrides a default

Suppose default tags are [stable], and a config explicitly sets tags: []. Under replacement semantics, the result must be the empty list, not the default. Therefore the resolver must distinguish “config omitted tags” from “config supplied an empty list.”

2.11 Aliases as a rewrite system

Let Σ^* be token sequences.

Definition 2.15 (alias rewrite). Each macro alias defines a rewrite

$$lx \rightarrow rx,$$

where l is the alias command path and r is the target path followed by fixed prefix tokens.

Compile-time law 2.2 (termination). Alias expansion must terminate. One sufficient policy is to require every alias target to be a primary command rather than another alias. A more flexible policy permits alias-to-alias targets but rejects cycles in the alias graph.

Compile-time law 2.3 (determinism). At a given trie node, no token may select both a primary child and an alias, and no two aliases may share the same path.

2.11.1 Worked example: precedence of injected and caller options

Alias:

```
find -> note search --format=jsonl --limit=10
```

Invocation:

```
find "typed parsers" --limit=20
```

After expansion:

```
note search --format=jsonl --limit=10 "typed parsers" --limit=20
```

Under RepeatLast, the caller overrides the alias. Under RepeatFirst, the alias owns the value. Under RepeatReject, the alias is unsafe unless caller duplicates are diagnosed. This is a user-facing policy and should be declared in the field binding or alias contract, not inherited accidentally from pflag.

2.11.2 Counterexample: textual expansion after parsing

If the caller’s tokens are parsed first and alias flags are merged later as a higher-priority source, the result may reverse intended precedence. Alias expansion should occur at a clearly documented phase, ideally before CLI occurrence semantics are applied, while preserving provenance that identifies injected tokens.

2.12 Completion is parsing the residual language

Shell completion is often implemented as a separate collection of string-prefix checks. That duplicates grammar decisions. A stronger view begins with the accepted invocation language $L \subseteq \Sigma^*$.

Definition 2.16 (residual language). For a typed prefix w , the residual language is

$$D_w(L) = \{x \in \Sigma^* \mid wx \in L\}.$$

The valid next tokens are prefixes of words in $D_w(L)$. This is analogous to Brzozowski derivatives for regular expressions and derivative-based parsing (Owens, Reppy, and Turon 2009; Might, Darais, and Spiewak 2011). A full CLI language is not necessarily regular because dynamic value completers may consult state, but the residual idea still supplies the architecture: run the same parser in partial mode, then ask the resulting state what can follow.

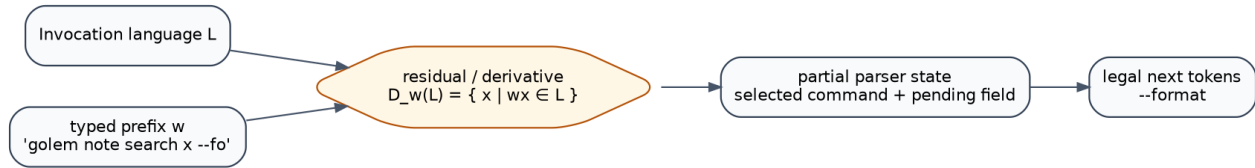


Figure 2.5: Completion as the residual language after a valid typed prefix.

Definition 2.17 (partial parse state). A partial parse returns either a diagnostic or a state containing:

- current command node;
- whether options are still recognized;
- which semantic fields have occurrences;
- the next positional slot;
- an optional field whose value is currently expected;
- the incomplete final token prefix.

2.12.1 Worked example: completing an option name

Prefix:

golem note search query --fo

The partial parser selects note search, binds query, remains in option-recognition mode, and sees partial token --fo. The effective option set contains --format, so the residual interpreter returns:

```
[{"value": "--format", "kind": "option", "description": "output framing"}]
```

2.12.2 Worked example: completing a choice value

Prefix:

golem note search query --format j

The partial state is Expect(output.format) with incomplete value prefix j. The choice codec declares values table, json, jsonl, csv, and tsv. Completion returns json and jsonl.

2.12.3 Worked example: dynamic completion

A field may declare a completer:

```
type Completer interface {
    Complete(ctx context.Context, q CompletionQuery) ([]Candidate, error)
}
```

The query carries semantic command and field identities, already-resolved safe inputs, and cancellation. It should not receive an arbitrary mutable parser object. Dynamic completion remains advisory and should be bounded, side-effect-light, and forbidden from printing protocol data to stdout.

2.12.4 Counterexample: completion that calls normal execution

A partially typed command is intentionally invalid. Calling the normal executor may trigger required validation, configuration loading, network initialization, or effects. Completion needs a partial interpretation mode, not a normal execution followed by error recovery.

2.13 Structured diagnostics

Strings such as `unknown flag: --foramt` are useful to humans and useless to other interpreters. Modern CLIs are consumed by shells, editors, agents, tests, and UIs. Diagnostics should be data.

Definition 2.18 (diagnostic). A diagnostic is a record

$$d = (c, m, p, f, [a, b], E, S, N),$$

where:

- c is a stable code;
- m is a human message;
- p is command path;
- f is optional field identity;
- $[a, b]$ is a token or character span;
- E is an expected set;
- S is a suggestion set;
- N is structured notes and provenance.

A terminal renderer may format:

```
error[E_UNKNOWN_FLAG] at `golem note search`
  token 4: --foramt
            ^^^^^^^
  unknown option
  did you mean: --format
```

A JSON renderer can preserve the same semantics without scraping the string.

2.13.1 Diagnostic taxonomy

A useful first taxonomy is:

Layer	Example codes
compile	E_FLAG_COLLISION, E_ALIAS_TARGET, E_POSITIONAL_GRAMMAR
scan	E_UNKNOWN_FLAG, E_MISSING_VALUE, E_SHORT_CLUSTER
decode	E_TYPE, E_CHOICE, E_FILE_READ
resolve	E_SOURCE_FIELD, E_SOURCE_CONFLICT
validate	E_REQUIRED, E_CONSTRAINT
execute	E_COMMAND, E_OUTPUT_CLOSE, E_CANCELLED

Design law 2.4 (diagnostic secrecy). Diagnostic messages, JSON, traces, manifests, and help must not reveal secret values. A Secret field may expose presence and source but should redact raw and parsed values.

2.14 POSIX compatibility and intentional language choices

POSIX utility syntax guidelines provide a valuable portability baseline: options precede operands, `--` terminates options, option names are predictable, and option arguments are separated consistently (The Open Group 2024b, 2024a). Modern CLIs also commonly support long options, interspersed options, subcommands, negated Boolean names, and clustered short flags.

The native parser should define a **compatibility profile**, not claim universal POSIX conformance. Suggested defaults are:

- accept long options `--name value` and `--name=value`;
- accept one-rune short options and Boolean clusters;
- permit a valued short option to consume the remaining cluster or next token;
- support `--` exactly;
- enable interspersed options for compatibility with current Cobra behavior, but allow a command to disable it;
- do not enable command-prefix matching by default;
- treat case sensitivity as fixed and documented;
- reserve explicit policies for repeat handling, negation, comma splitting, and unknown-option forwarding.

A wrapper or plugin command may use a special forwarding grammar:

```
type UnknownOptionPolicy int
const (
    RejectUnknown UnknownOptionPolicy = iota
    ForwardAfterTerminator
    ForwardAllAfterFirstOperand
)
```

This is safer than a single `DisableFlagParsing` Boolean because it states which syntax remains owned by the framework.

2.15 Formal properties worth testing

The design does not require a proof assistant before implementation, but it benefits from executable laws.

Property 2.1 (surface-form coherence). Equivalent CLI spellings for one binding decode to equal semantic values.

Property 2.2 (catalog determinism). Compiling the same authored program produces byte-equivalent manifests and identical diagnostic ordering.

Property 2.3 (parse determinism). For fixed catalog and token sequence, parsing returns one invocation or one deterministic diagnostic.

Property 2.4 (source determinism). For fixed ordered source assignments and policies, resolution returns one value assignment and provenance trace.

Property 2.5 (required-after-merge). Required validation depends on the resolved domain, not the set of CLI-changed flags.

Property 2.6 (completion soundness). Every static candidate returned by completion can extend the current prefix without producing an immediate unknown-name diagnostic.

Property 2.7 (help-reference coherence). Every option rendered in help maps through the effective binding relation to a known semantic field.

Property-based testing in the style of QuickCheck (Claessen and Hughes 2000) is well suited to token grammars. Generate small well-formed catalogs, generate valid invocations from them,

render alternative surface forms, and assert semantic equivalence. Generate invalid catalogs and assert that compilation rejects rather than partially mounts them.

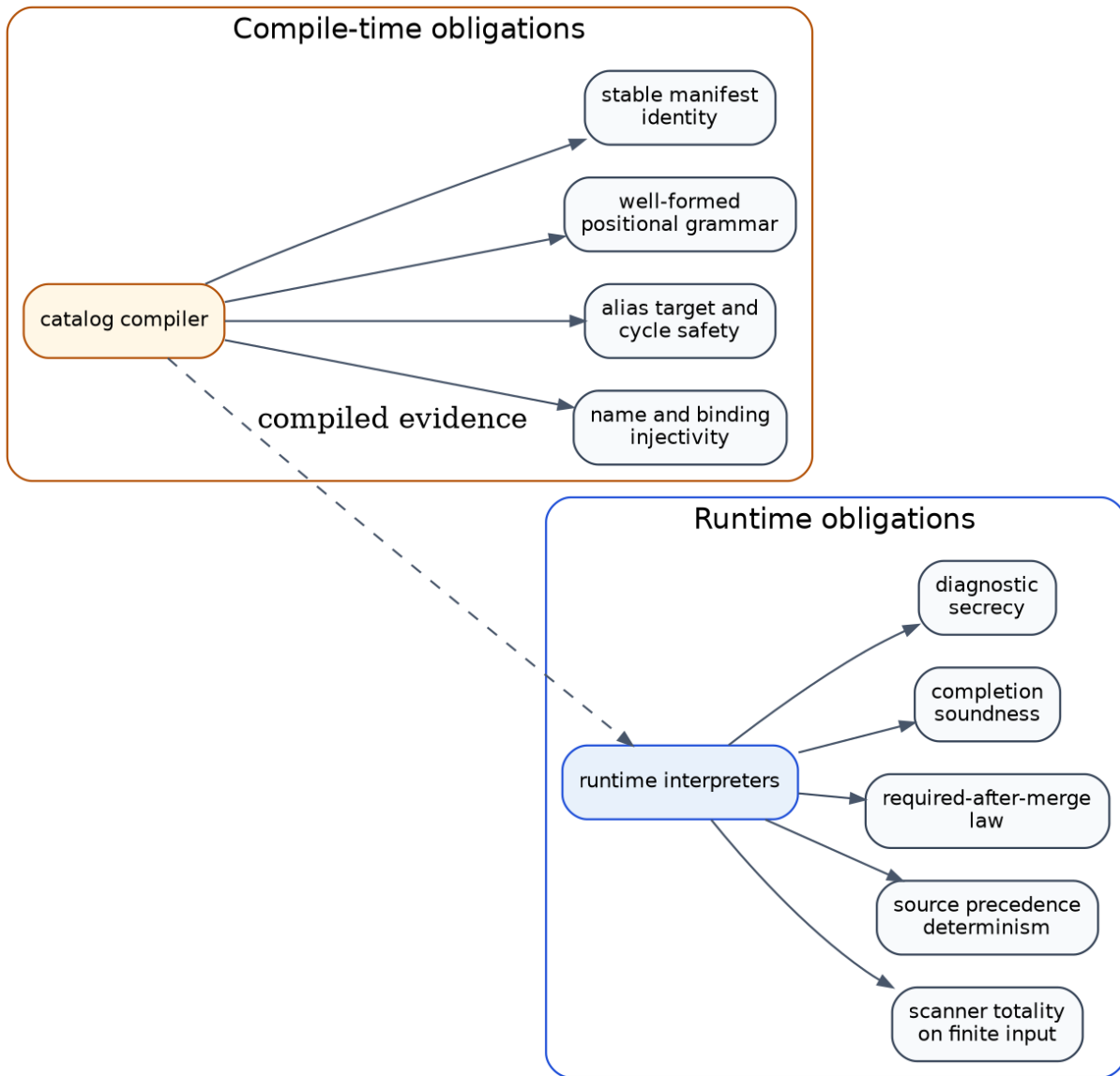


Figure 2.6: Compile-time and runtime proof obligations for the proposed native runtime.

2.16 Chapter synthesis

The mathematical model produces a practical architecture:

1. commands form a finite prefix language represented by a trie;
2. semantic fields form a typed signature;
3. CLI bindings are injective relations from names to field identities;
4. compilation computes effective scopes and rejects global inconsistencies;
5. scanning produces raw occurrences and a trace;
6. semantic codecs produce typed source assignments;

7. ordered partial-map folds resolve values and provenance;
8. validation runs on the resolved assignment;
9. aliases are validated rewrites;
10. completion interprets partial parser states;
11. diagnostics are structured results.

This is enough structure to replace Cobra without inventing a second Glazed.

2.17 Exercises for Chapter 2

1. Build the command trie for `golem` and mark structural namespaces versus runnable nodes. What should happen when the user invokes only `golem note`?
2. Define semantic IDs and CLI bindings for two database connections, `source` and `destination`, each with fields `host`, `port`, and `password`. Show the effective long names.
3. Prove binding injectivity is necessary for deterministic scanning. Construct a counterexample in which two fields share `-v`.
4. Extend the positional grammar with an “exactly one of two shapes” alternative. Explain why deterministic binding may require subcommands or an explicit separator instead.
5. Trace the scanner state for `golem -v note search query -n5 --tag a,b -- --format`.
6. Decide how the token `-1` is interpreted in each state: command mode, expecting integer value, operand-only mode, and after a variadic string operand under non-interspersed policy.
7. Give a pointwise proof that right-biased override is associative. Is it commutative? Supply a counterexample.
8. Define a merge operator for set-valued fields. Is it associative? How should provenance record duplicate contributions?
9. An alias injects `--tag stable`, and the caller supplies `--tag experimental`. Compute the result under source replacement, within-source concatenation, and cross-source append.
10. Construct an alias cycle and describe two compile-time algorithms for detecting it.
11. For prefix `golem cache clear --`, compute the residual static candidates. Should option names be returned?
12. Design a structured diagnostic for an invalid choice that came from an environment variable rather than a CLI token.
13. State a policy for abbreviated long options. Show how adding a new flag can turn a previously unique abbreviation into an ambiguity.
14. Compare `--bool=false`, `--no-bool`, and `--bool false`. Which forms should the native grammar accept, and why?
15. Design a property generator that emits valid option clusters. State the oracle used to check the parser.
16. Explain why a manifest should expose semantic IDs and bindings separately.

Chapter 3

Architecture and Experimental Implementation

3.1 Learning objectives

After this chapter, the reader should be able to:

- translate the formal model into Go package and API boundaries;
- explain the purpose of a catalog compiler and immutable runtime indexes;
- evaluate a native parser prototype without confusing feasibility with readiness;
- interpret the reported tests, static audit, and microbenchmarks;
- derive implementation obligations for completion, manifests, help, and execution;
- identify the prototype's limitations and the experiments still required.

3.2 Motivation: experiment on the architecture, not only the scanner

A twenty-line option loop can demonstrate that strings may be split at `=`. It cannot answer the architectural questions in this thesis. The experiment must exercise the complete proposal:

- whole-program command compilation;
- command and binding collision checks;
- effective inherited option scopes;
- structured parser states and diagnostics;
- raw occurrence collection;
- semantic decoding;
- source precedence and provenance;
- required validation after merging;
- alias expansion;
- completion from partial state;
- help and manifest generation from the same catalog.

The experimental prototype is included in the source bundle. It uses only the Go standard library and does not import Glazed or Cobra. This isolation has two purposes. First, it proves that the proposed runtime does not secretly depend on pflag behavior. Second, it prevents the prototype from claiming integration that has not been tested. The design must later be adapted to Glazed's full field codecs and values packages.

3.3 Target architecture

The target has two major phases: compilation and interpretation.

```

authored CommandSet
  -> CompileCommandSet
  -> immutable CommandCatalog
    -> native Parse
    -> Complete
    -> RenderHelp
    -> Describe / Manifest
    -> transitional Cobra adapter

```

Parse result + config/env/provided sources

```

  -> ResolveValues with provenance
  -> Validate
  -> ExecutionPlan
  -> existing cmds/runner

```

The crucial dependency direction is:

authoring model → catalog → frontends,

not

authoring model → Cobra → help/completion/schema facts.

3.4 Proposed package decomposition

Figure 3.1 presents one possible package graph.

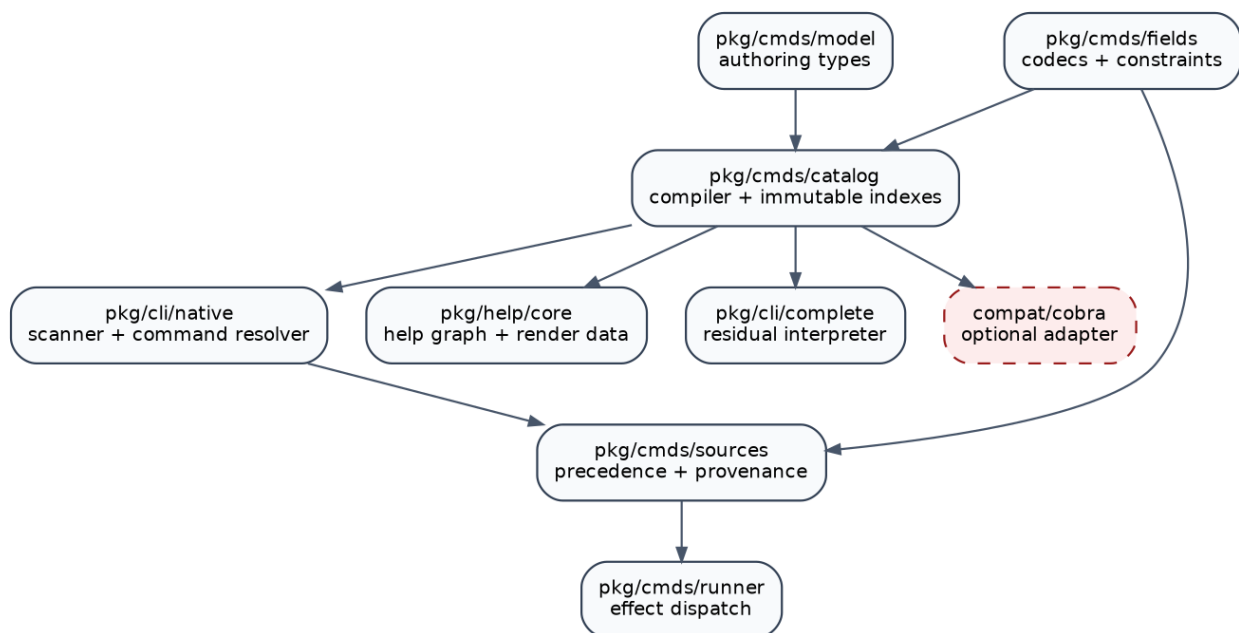


Figure 3.1: Proposed package dependencies after the native catalog is introduced.

The exact package names may change, but the acyclic responsibilities matter.

3.4.1 `pkg/cmds/model`

Owns authoring types:

- command descriptions and paths;
- command contracts;
- semantic field specifications;
- section instances;
- frontend binding declarations;
- alias declarations;
- authored examples and help references.

This package must not import Cobra, pflag, terminal UI packages, or config loaders.

3.4.2 `pkg/cmds/catalog`

Owns:

- whole-program normalization;
- path trie construction;
- effective-scope calculation;
- binding indexes;
- positional grammar checks;
- alias resolution;
- deterministic diagnostics;
- immutable catalog snapshots and stable manifests.

3.4.3 `pkg/cmds/fields`

Retains semantic types, codecs, validity checks, secret rendering policy, and structured constraints. Cobra-specific methods move out.

3.4.4 `pkg/cli/native`

Owns the concrete token grammar, scanner state machine, raw occurrences, command selection, and parser traces.

3.4.5 `pkg/cmds/sources`

Retains defaults, files, environment, maps, and custom source middleware. It gains a native CLI source that consumes parser occurrences by semantic field ID rather than reading a Cobra command.

3.4.6 `pkg/cli/complete`

Interprets partial parser states and field completers. Shell-specific scripts remain adapters over a stable machine protocol.

3.4.7 `pkg/help/core`

Joins model-derived reference nodes with authored help sections. Rendering and storage remain separate packages.

3.4.8 compat/cobra

During migration, compiles a CommandCatalog into Cobra commands. It is explicitly downstream of the catalog. After first-party migration, it can move to an optional module so users who need Cobra interoperability can retain it without imposing the dependency on every Glazed application.

3.5 Catalog API design

A production catalog should expose immutable interfaces rather than raw maps.

```
type CommandID string
type FieldID string

type CommandCatalog interface {
    Program() ProgramInfo
    Root() CommandView
    LookupCommand(path []string) (CommandView, bool)
    ResolveCommand(tokens []string, policy ResolvePolicy) ResolveResult
    Manifest() Manifest
    Fingerprint() [32]byte
}

type CommandView interface {
    ID() CommandID
    Path() []string
    PrimaryName() string
    Names() []CommandName
    Runnable() bool
    Children() []CommandView
    EffectiveFields() []FieldView
    Positionals() []FieldView
    LookupLong(string) (FieldView, bool)
    LookupShort(rune) (FieldView, bool)
    Contract() CommandContract
    Source() SourceLocation
}
```

Immutability is practical rather than metaphysical: slices returned by accessors are copies or read-only views; internal maps are not exposed; and catalog construction finishes before concurrent parsing. The authored model can remain convenient and mutable while loading plugins or applying options. Compilation is the ownership boundary.

Definition 3.1 (catalog fingerprint). A catalog fingerprint is a digest of the normalized, public semantic contract. It excludes process addresses, nondeterministic map order, secret values, and transient runtime state.

A fingerprint supports:

- documentation provenance;
- completion-cache invalidation;
- compatibility snapshots;
- agent contracts;
- differential-test fixture identity.

It is not authorization and not a binary provenance proof unless build identity is included separately.

3.6 Field bindings as compiled evidence

The current Definition type combines semantic and CLI-facing names. A non-breaking transitional API can add explicit bindings while deriving a default binding from legacy fields.

```
type FieldSpec struct {
    ID          FieldID
    Name        string
    Type        fields.Type
    Required     RequiredPolicy
    Default      *fields.RawValue
    Choices     []fields.RawValue
    Description string
    Secret       bool
}

type BindingSet struct {
    CLI    *CLIBinding
    Env    []EnvBinding
    Config []ConfigBinding
    JSON   []JSONBinding
}

type CLIBinding struct {
    Long          []string
    Short         []rune
    Metavar       string
    Repeat        RepeatPolicy
    Split         SplitPolicy
    Negation      NegationPolicy
    Visibility     Visibility
    InterspersedOverride *bool
}
```

A legacy constructor can compile:

```
fields.New("api_token", fields.TypeSecret,
    fields.WithShortFlag("t"),
)
```

into semantic ID `default.api_token` and default CLI binding `--api-token, -t`. New code can override the binding explicitly. This allows migration without forcing every downstream repository to rewrite field declarations immediately.

3.6.1 Worked example: detecting collisions before command mount

The compiler computes effective fields for each command and then indexes every long and short binding. If `logging.verbose` and `command.verbose` both claim `-v`, it emits both source locations:

```
{
  "code": "E_SHORT_COLLISION",
  "path": ["note", "search"],
  "binding": "-v",
  "claims": [
    {"field": "logging.verbose", "source": "logging.NewSection"},
    {"field": "default.verbose", "source": "search.go:42"}
  ]
}
```

This is more actionable than a pflag panic after part of the tree has been built.

3.7 Parse API design

The parser should expose lexical facts and avoid loading configuration or executing commands.

```
type ParseRequest struct {
    Args      []string
    Trace      bool
    Partial    bool
    StartPath []string
}

type ParseResult struct {
    Command      CommandID
    CanonicalPath []string
    InvocationPath []string
    Occurrences  map[FieldID][]Occurrence
    Operands     []Operand
    Trace         []TraceStep
    EndOfOptions bool
    Residual      *ResidualState
}

func Parse(
    ctx context.Context,
    catalog CommandCatalog,
    request ParseRequest,
) (ParseResult, *Diagnostic)
```

Returning a diagnostic value rather than only error lets callers choose terminal, JSON, TUI, or test rendering. The ordinary Go error interface can still be implemented for convenience.

3.7.1 Scanner and semantic decoder boundary

```
// Scanner responsibility:
--limit=5 -> Occurrence{Field: "default.limit", Raw: "5"}

// Field codec responsibility:
"5"      -> int(5)
```

This boundary preserves Glazed's existing field semantics and allows the CLI grammar to evolve independently.

3.8 Resolution API design

A native CLI source should fit the current source model.

```
type SourceContribution struct {
    Source      SourceID
    Field        FieldID
    Raw          []string
    Metadata     map[string]any
}

type Resolver interface {
```

```

    Resolve(
        ctx context.Context,
        schema SchemaView,
        sources ...Source,
    ) (*values.Values, []Diagnostic)
}

```

The CLI source converts occurrences into `SourceContribution` records. Existing config and environment sources continue to work. Required validation becomes a final validator over `values.Values`.

A valuable extension is to represent the resolution plan before executing it:

```

type ResolutionPlan struct {
    Sources []PlannedSource
    Policy  PrecedencePolicy
}

```

Help can then explain not only which fields exist but where the runtime will look for them.

3.9 Command contracts

The current command kind is inferred through Go interfaces. That is useful for execution but incomplete for help and machine consumers. A command contract can state effect and output semantics explicitly.

```

type EffectClass string
const (
    EffectReadOnly    EffectClass = "read-only"
    EffectLocalWrite  EffectClass = "local-write"
    EffectRemoteWrite EffectClass = "remote-write"
    EffectProcess     EffectClass = "process-control"
)

type OutputContract struct {
    Mode      string      // none, text, rows, stream
    Formats   []string
    Default   string
    Schema    *JSONSchemaRef
    Streaming bool
}

type CommandContract struct {
    Effect      EffectClass
    Idempotency string
    Output      OutputContract
    Cancellation bool
    Experimental bool
}

```

This metadata must be honest. Declaring read-only does not enforce it; it creates a reviewable and testable promise. Enforcement remains in the effect-owning implementation and surrounding authorization policy.

3.9.1 Worked example: describe versus plan

`golem note search` has a static contract:

```
{
  "effect": "read-only",
  "output": {
    "mode": "rows",
    "formats": ["table", "json", "jsonl", "csv", "tsv"],
    "streaming": true
  }
}
```

A concrete invocation has a plan:

```
{
  "command": "note.search",
  "query": "snapshot cuts",
  "resolvedFormat": "jsonl",
  "resolvedLimit": 5,
  "sources": {
    "output.format": "alias:find",
    "default.limit": "cli"
  },
  "effect": "read-only"
}
```

The first is model-derived and safe to cache. The second depends on values, aliases, profiles, and configuration.

3.10 Help as a typed intermediate representation

A modern help system should not format directly from internal structs. It should first build a renderer-neutral document model.

```
type HelpDocument struct {
  Subject      HelpSubject
  Summary      []Block
  Usage        UsageModel
  Commands     []CommandEntry
  Sections     []FieldSection
  Examples     []Example
  Related      []HelpLink
  Diagnostics  []Diagnostic
}

type Block interface{ helpBlock() }
type Paragraph struct{ Inlines []Inline }
type DefinitionList struct{ Items []DefinitionItem }
type Callout struct{ Kind, Title string; Body []Block }
type CodeBlock struct{ Language, Text string }
```

Renderers can produce:

- concise terminal help;
- long Markdown help;
- man pages;
- JSON for agents and editor integrations;
- Bubble Tea views;
- web pages;
- test snapshots.

This resembles compiler intermediate representations: one normalized document tree supports several targets. It also resembles the separation between syntax and presentation in language-server protocols (Microsoft 2024).

3.11 Joining reference help and authored help

Glazed’s help section model already contains `Slug`, `SectionType`, package metadata, title, content, topics, flags, commands, and display options. The proposed help graph links these sections to stable catalog identities.

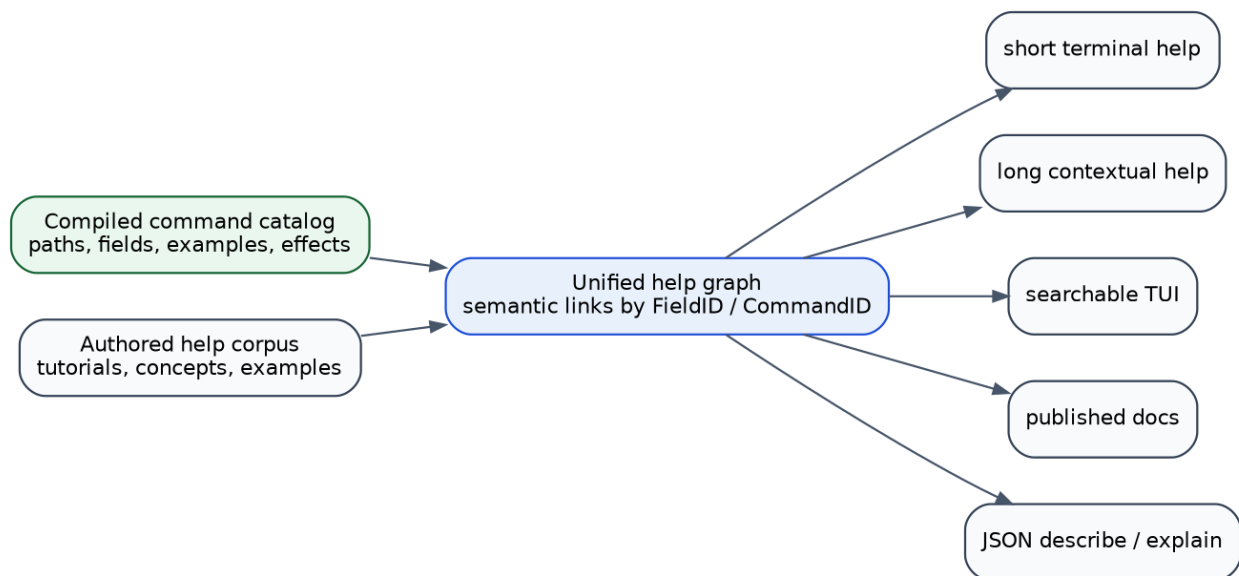


Figure 3.2: A unified help graph combines exact catalog facts with authored tutorials and examples.

Definition 3.2 (help graph). A help graph is a labeled graph

$$G_H = (N, E),$$

whose nodes include commands, fields, sections, examples, tutorials, concepts, and external references. Edges include documents, example-of, related-to, replaces, requires, and see-also.

A search query can return both exact command facts and conceptual articles. Contextual help can follow edges from the selected command and fields actually present in the invocation.

3.11.1 Worked example: contextual error help

Invalid invocation:

```
golem cache clear --all --key x
```

A structured constraint diagnostic identifies fields `cache.all` and `cache.key`. The help graph can attach:

- the exact rule: mutually exclusive;
- the command’s usage;
- an authored example of clearing one key;

- a concept note explaining destructive operations;
- a suggested corrected invocation.

The system does not need to search raw Markdown for strings `all` and `key`; it follows semantic IDs.

3.12 The experimental prototype

The included module `thesis.local/glazednative` implements a deliberately narrow version of the architecture.

3.12.1 Implemented features

- mutable authoring program with commands, global fields, and macro aliases;
- whole-program compiler producing a command trie and effective option indexes;
- collision checks for command edges, aliases, semantic IDs, long names, and short names;
- positional-grammar checks for required, optional, and variadic fields;
- long options with separate `=` values;
- Boolean negation using `--no-name`;
- clustered short Boolean options and attached valued options;
- exact `--` terminator behavior;
- interspersed options;
- alias expansion with a fixed token prefix;
- raw occurrence maps keyed by semantic ID;
- scalar repeat policies;
- typed decoding for strings, secrets, integers, floats, Booleans, lists, and choices;
- ordered source resolution with per-field provenance logs;
- required validation after source merging;
- residual-state completion for commands, options, and choice values;
- Markdown help rendered from the catalog;
- versioned JSON manifest with secret defaults removed;
- structured diagnostics with codes, paths, fields, token indexes, expected values, and suggestions.

3.12.2 Representative authored fixture

```
func ExampleProgram() Program {
    return Program{
        Name:      "golem",
        Interspersed: true,
        Global: []Field{
            {Name: "config", Short: 'c', Kind: String, Inherited: true},
            {Name: "verbose", Short: 'v', Kind: Boolean, Inherited: true},
        },
        Commands: []Command{
            {
                Path:      []string{"note", "search"},
                Runnable: true,
                Fields: []Field{
                    {Name: "query", Positional: true,
                     Required: true, Kind: String},
                    {Name: "limit", Long: "limit", Short: 'n',
                     Kind: Integer, Default: []string{"20"}},
                    {Name: "format", Section: "output",
                     Kind: Choice,
```

```

        Choices: []string{"table", "json", "jsonl", "csv", "tsv"},
        Default: []string{"table"}},
    {Name: "tag", Kind: StringList,
     Merge: MergeAppend},
    },
},
Aliases: []Alias{
    {Path: []string{"find"},
     Target: []string{"note", "search"},
     Prefix: []string{"--format=jsonl"}},
},
}
}

```

3.12.3 Representative resolution

The demonstration invokes:

```
golem find "prefix cuts" --limit 5 --tag architecture
```

with config source tag=[research]. The result contains canonical command path note search, invocation path find, alias-injected output.format=jsonl, limit=5, tags [research, architecture], and a transition trace.

The full JSON is included as experiments/demo-resolution.json.

3.13 Experimental questions

The prototype addresses four limited questions.

EQ1 - Can whole-program compilation catch effective-scope errors without Cobra?

Yes for the implemented collision and positional checks. Tests deliberately introduce an inherited/application --verbose collision and assert a compile diagnostic.

EQ2 - Can one parser state machine support normal parsing and completion?

Yes for static commands, options, and choices in the prototype. Completion runs the scanner in partial mode and inspects its residual state.

EQ3 - Can native CLI occurrences participate in source provenance without special cases?

Yes. Parser output is converted to a cli source after config and environment sources. The resolver appends provenance steps and validates required fields afterward.

EQ4 - Is the architecture obviously too slow for ordinary CLIs?

No under the synthetic benchmark. Compilation is millisecond-scale and intended to occur once. Parse and completion operations are microsecond-scale. This does not establish performance on real Glazed commands, dynamic completers, filesystem-backed fields, or very large plugin catalogs.

3.14 Validation results

The rebuilt experiment was executed with:

```
go test ./... -race -cover -count=1
go vet ./...
```

```
go test -bench=. -benchmem -run '^$' -count=5 ./...
```

The core package passes its tests under the race detector with 73.6 percent statement coverage. `go vet` reports no findings. The command demonstration has no dedicated test coverage, so module-wide output also reports zero coverage for that small package.

The tests cover:

- compile-time effective flag collisions;
- long options, short clusters, attached values, aliases, and `--`;
- source precedence and provenance;
- required values supplied by non-CLI sources;
- residual option and choice completion;
- help and manifest interpretation;
- structured unknown-option suggestions.

The tests do not cover every grammar branch or every field type. Coverage is evidence that the prototype is executable, not evidence that the grammar is complete.

3.15 Microbenchmark results

The benchmark catalog contains 100 commands arranged under ten groups. Each command has forty string options and one required positional. The parse workload selects one command and supplies two options. The completion workload completes a long-option prefix.

Benchmark	Median	Allocated bytes/op	Allocations/op
Compile catalog	3.407 ms	about 7.01 MB	about 7,373
Parse invocation	1.567 microseconds	1,992 B	22
Complete prefix	5.861 microseconds	5,160 B	62

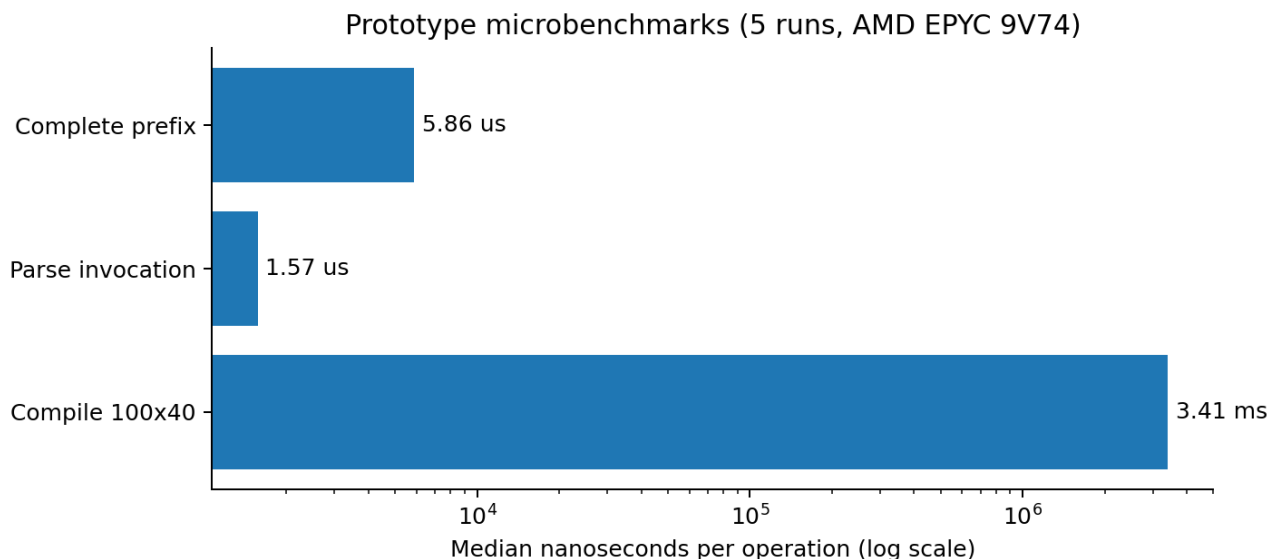


Figure 3.3: Prototype microbenchmarks. The horizontal axis is logarithmic; labels give the measured median units.

3.15.1 Interpretation

The compilation cost is dominated by normalization, map construction, copied field views, and diagnostics-friendly structures. It occurs once per catalog assembly. A production compiler should reduce allocations, but the measured cost is already negligible for ordinary CLI startup relative to config loading, dynamic plugin discovery, or network initialization.

Parsing allocates because the prototype copies token slices, stores traces unconditionally, creates occurrence objects, and returns maps. A production parser could make traces optional, pool temporary buffers cautiously, and pre-size maps from catalog metadata. Premature optimization is unnecessary until a real Glazed corpus is benchmarked.

Completion is several times slower than parsing because it constructs candidates and sorts them. It remains far below typical shell-process startup and dynamic-completion latency. The important future measurement is end-to-end completion time in real binaries, including process startup and configured completers.

Experimental caution. Microbenchmarks on one synthetic catalog do not compare the prototype with Cobra, do not measure help rendering, and do not establish tail latency. They answer only whether the proposed architecture is plausibly affordable.

3.16 Static audit of the existing native parser seed

The included script `experiments/audit_existing_string_parser.py` models one condition visible in the pinned `GatherFlagsFromStringList` source:

```
rawValues[observedFlagName] = ...
...
if param.Required {
    _, ok := rawValues[param.Name]
}
```

The audit compares semantic names with observed short, normalized, and prefixed keys. It demonstrates that equality is not invariant under binding transformations. This is not a runtime import of Glazed and is not presented as a failing upstream test. It is a mechanized counterexample extracted from the source logic.

The appropriate upstream experiment is a table-driven test added beside `strings_test.go`:

```
{
    name: "required field satisfied through short flag",
    args: []string{"-r", "value"},
    definition: &Definition{
        Name: "required", ShortFlag: "r",
        Type: TypeString, Required: true,
    },
    want: "value",
}
```

Equivalent cases should cover underscore normalization and section prefixes.

3.17 Compatibility matrix

A native replacement should publish a grammar matrix before implementation is declared compatible.

Feature	Current evidence	Prototype	Proposed production policy
--name value	Cobra and string parser	yes	yes
--name=value	Cobra and string parser	yes	yes
one-rune short options	Cobra; Glazed permits string ShortFlag values	yes	yes; reject multi-rune short names at compile time or define them explicitly
Boolean short clusters	Cobra/pflag behavior	yes	yes for Boolean prefixes
valued attached short -n5	pflag-compatible expectation	yes	yes, documented
-- terminator	Cobra; TODO in string parser	yes	yes
interspersed options	Cobra configurable behavior	yes	catalog/command policy
unknown-option forwarding	wrapper commands use disable parsing patterns	no	explicit forwarding policy
flag normalization	Cobra/pflag normalization hooks; Glazed underscores to hyphens	limited	one compiled normalization policy
dynamic completion	Cobra functions	no	semantic completer API
file/directory completion	Cobra annotations/directives	no	typed completion hints
required from env/config	current source architecture	yes	post-merge validation
repeated scalar policy	pflag generally last value visible	explicit	per-binding explicit policy
list cross-source merge	Glazed merge semantics depend on source behavior	explicit	per-field replace/append/set-union
deprecation warnings	Cobra/pflag support	no	binding metadata + structured diagnostic
help/version control paths	Cobra lifecycle	no	native control commands/options compiled into root policy

The matrix should become executable tests, not remain a document.

3.18 Differential and metamorphic testing

A migration cannot rely solely on unit tests written for the new implementation. It needs oracles.

3.18.1 Differential testing

Run the same compiled Glazed command declarations through:

1. current Cobra adapter;

2. native parser and resolver.

Compare:

- selected canonical command;
- semantic field values;
- source provenance where comparable;
- returned error category;
- stdout/stderr/exit status at the process boundary;
- completion candidates;
- help reference facts.

Differential testing is an established technique for finding discrepancies without a complete independent oracle (McKeeman 1998). Differences are then classified as bugs, intentional language changes, or unspecified behavior that must be specified.

3.18.2 Metamorphic properties

Even when Cobra cannot serve as oracle, transformations can preserve meaning:

```
--limit 5          <-> --limit=5
--verbose          <-> --verbose=true
--tag a --tag b    <-> --tag=a,b      (only under the declared split policy)
primary path      <-> name alias      (without macro defaults)
config omitted    <-> absent map entry
```

A metamorphic test applies the transformation and asserts equal resolved semantic values and equivalent effects.

3.18.3 Golden compatibility corpus

The corpus should include real invocations collected from first-party repositories, with secrets redacted. Each fixture contains:

```
name: note-search-jsonl
command: [note, search]
argv: [note, search, snapshot cuts, --format, jsonl]
env: {}
config: {}
expect:
  command_id: note.search
  values:
    default.query: snapshot cuts
    output.format: jsonl
  error: null
```

Fixtures should also cover failures, aliases, help, completion, and cancellation.

3.19 Prototype limitations

The prototype is intentionally incomplete. Important omissions are:

1. It reimplements only a subset of Glazed field types. File-derived, date, object, key-value, and object-list codecs are not integrated.
2. It uses a simple authored Program rather than compiling actual cmds.Command values and schemas.
3. It has no dynamic plugin loading, help corpus, Bubble Tea integration, or shell script generators.

4. Alias expansion rewrites a token slice and preserves only coarse token indexes; a production source map should track original and synthetic spans separately.
5. Effective inheritance is simplified and does not yet support deliberate local shadowing by semantic identity.
6. The manifest serializes authoring declarations rather than a fully normalized public catalog with stable IDs and source coordinates.
7. Secret redaction is limited to secret defaults in the manifest. Diagnostics and provenance need a comprehensive secrecy policy.
8. Configuration and environment parsing are represented as already-collected source maps. The prototype does not parse files or environment names.
9. Completion is static and has no file, directory, remote, or application-defined completers.
10. Error recovery stops at the first runtime parse error. A compiler should aggregate independent static errors, while a single invocation usually reports the most relevant parse error.
11. The benchmark does not include Cobra and therefore is not a performance comparison.
12. The API has not been tested against downstream Glazed repositories.

These limits are not incidental. They define the next research and engineering tasks.

3.20 Refinement obligations

The production implementation should be treated as a refinement of the formal model and current Glazed semantics.

Obligation 3.1 (field coherence). Every CLI occurrence resolves through a compiled binding to exactly one semantic field ID, and every typed value is produced by the field’s semantic codec.

Obligation 3.2 (source coherence). The native CLI source participates in the same precedence and provenance mechanisms as other sources.

Obligation 3.3 (command coherence). Parse, completion, help, manifest, and transitional Cobra mount use the same command path and availability indexes.

Obligation 3.4 (control-path safety). Help, version, describe, and completion do not trigger required-value validation or command effects.

Obligation 3.5 (error preservation). Command execution errors remain observable to the caller and process exit policy remains at the application boundary.

Obligation 3.6 (help truthfulness). Model-derived help reflects the same compiled catalog used by parsing; authored prose is linked but not allowed to redefine names or constraints.

Obligation 3.7 (secret non-disclosure). Secret raw values and typed values are excluded from public manifests, diagnostics, traces, help, completion, and default debug output.

3.21 Chapter synthesis

The experiment supports the architectural hypothesis. A single immutable catalog can drive parsing, source resolution, completion, help, and manifests without Cobra. The prototype is small enough to inspect and fast enough to justify integration experiments. Its failures and omissions sharpen the design: stable IDs, explicit bindings, post-merge validation, source maps for alias expansion, structured diagnostics, and a renderer-neutral help IR are not optional refinements. They are the mechanisms that prevent the native runtime from becoming another opaque parser adapter.

3.22 Exercises for Chapter 3

1. Design a `CompileCommandSet` signature that can return multiple diagnostics while preserving Go error conventions.
2. Specify which fields belong in a catalog fingerprint. Explain why descriptions may or may not belong, depending on the cache being invalidated.
3. Write pseudocode for compiling inherited fields while supporting explicit local shadowing by semantic ID.
4. Extend the prototype field binding to support a deprecated long name and a replacement. Where should the warning be generated?
5. Design an original-token source map for alias expansion. Show how a diagnostic for an injected invalid choice can mention both alias source and target field.
6. Add a `SetUnion` merge policy for list fields. Define deterministic output ordering.
7. Propose a public JSON schema for `Diagnostic`. Which fields are stable API and which are renderer hints?
8. The prototype completion benchmark allocates about 5 KB per operation. Identify three likely sources and two optimizations that preserve clarity.
9. Build a compatibility fixture for a command that forwards unknown options to a subprocess. Specify the expected boundary around `--`.
10. Design an integration test that proves `--help` works for a command with a required field supplied by no source.
11. Compare the responsibilities of `describe`, `explain`, and `plan`. Which operations can be pure?
12. Define a help intermediate representation that can render both a one-screen terminal summary and a long web page without losing semantic links.
13. Explain why the current Glazed help TUI should be adapted rather than replaced.
14. Propose a benchmark using actual Glazed command descriptions. What setup costs should be excluded from the parse loop?
15. Give a reason to keep the transitional Cobra adapter after the native parser is available, and a criterion for finally removing it.

Chapter 4

Migration, Modern Help, and Framework Evolution

4.1 Learning objectives

After this chapter, the reader should be able to:

- plan Cobra removal as a staged language migration;
- define compatibility profiles and classify behavioral differences;
- construct differential, property-based, and end-to-end evidence gates;
- evolve Glazed’s command framework without conflating authoring, compilation, parsing, resolution, and execution;
- design a modern help experience that serves humans, shells, editors, and agents from one semantic catalog;
- identify security, governance, and long-term research questions.

4.2 Motivation: dependency removal is a public-language migration

A direct dependency can be deleted in one commit. A language dependency cannot. Applications and users may rely on behavior they never named:

- options before or after subcommands;
- the exact handling of --;
- short clusters and attached values;
- aliases and hidden commands;
- when defaults are installed;
- whether help bypasses required validation;
- whether errors are printed, returned, or cause exit;
- completion ordering and file fallback;
- the division between local and inherited flags;
- the exact usage line and generated documentation.

Some of these behaviors are desirable contracts. Some are accidental Cobra compatibility. Some are bugs. Removing Cobra without classifying them converts all three categories into surprises.

Definition 4.1 (language migration). A language migration is a change to the accepted programs, their interpretations, diagnostics, or tooling of an interface language. For a CLI, the “programs” are token sequences plus external source assignments.

Definition 4.2 (compatibility class). A behavior difference belongs to one of four classes:

1. **preserve** - documented or widely depended-on semantics;
2. **repair** - a defect whose correction should be backported or clearly announced;
3. **intentional break** - a simplification or safety improvement requiring a major version;
4. **unspecified** - behavior that must be measured and then either specified or rejected.

The migration plan must attach evidence and a compatibility class to every observed difference.

4.3 A seven-phase migration

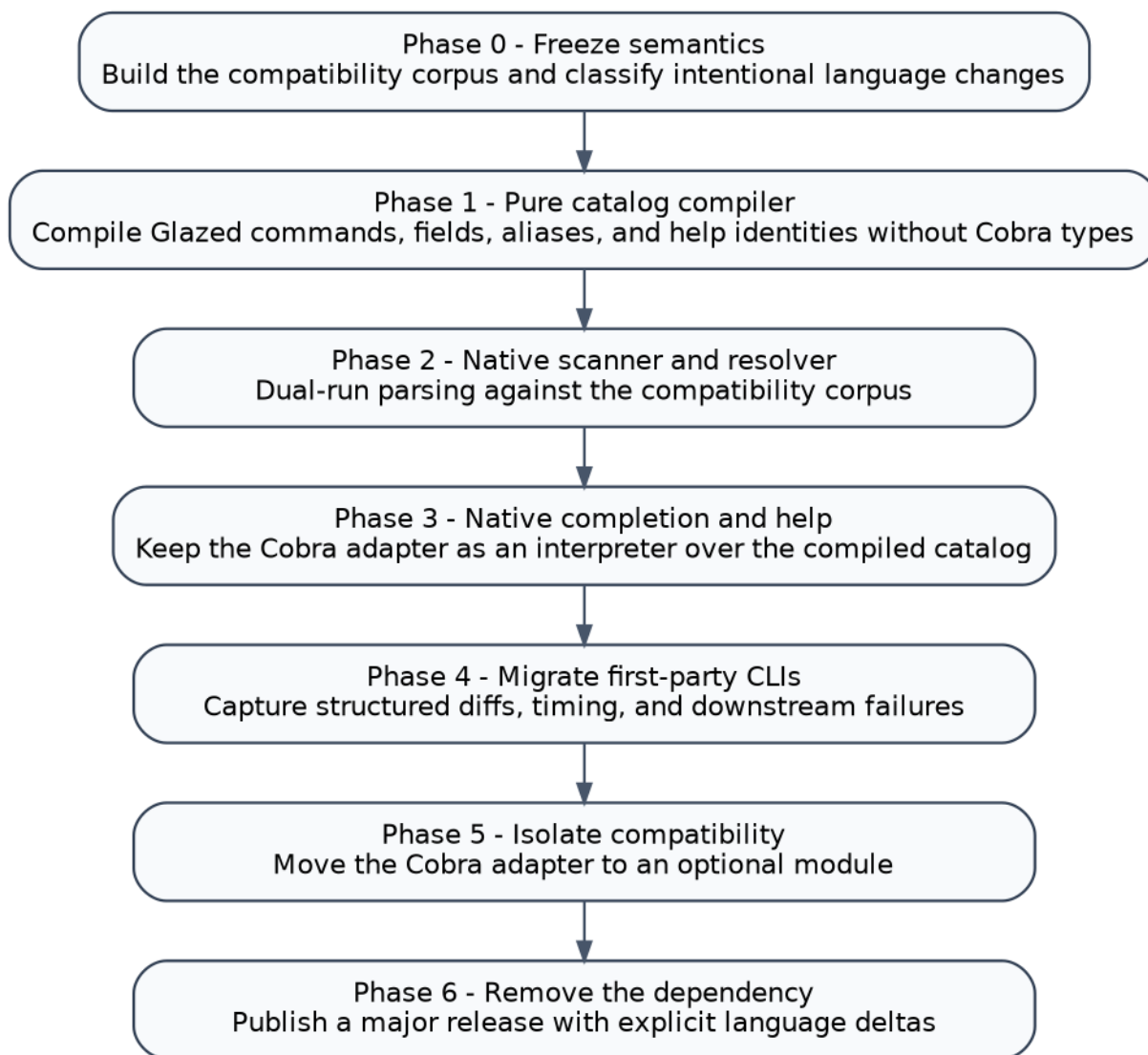


Figure 4.1: A staged migration from compatibility evidence to complete Cobra removal.

4.3.1 Phase 0 - Freeze and observe current semantics

Before adding a native parser:

- pin representative first-party command trees;

- export normalized command manifests from the current system;
- collect successful and failing argv fixtures;
- capture help, completion, returned errors, stdout, stderr, and exit status;
- add missing tests for the existing string parser, error propagation, aliases, source precedence, and output-field collisions;
- define secret-redaction procedures for fixtures.

The output is a versioned compatibility corpus, not a prose checklist.

4.3.2 Phase 1 - Introduce a pure catalog compiler

Move Cobra-free authoring concepts into a package that can compile:

```
catalog, diagnostics := catalog.Compile(commandSet)
```

The current Cobra adapter remains the only process frontend, but it now consumes the catalog instead of traversing mutable Glazed descriptions ad hoc. This phase yields immediate benefits:

- atomic collision validation;
- deterministic command ordering;
- stable command and field IDs;
- manifests for tests and documentation;
- a single place to calculate framework-injected inputs.

No user-facing parser behavior needs to change yet.

4.3.3 Phase 2 - Add the native scanner and resolver

Implement native parsing behind an explicit feature gate:

```
GLAZED_CLI_FRONTEND=native
```

or a construction option. Run both parsers in tests. For safe commands in development builds, the application may parse with both and log a redacted semantic diff while executing only the chosen result.

The native parser should initially target the preserve subset. Intentional changes remain disabled until separately approved.

4.3.4 Phase 3 - Add native completion and help reference rendering

Once parsing uses the catalog, completion should reuse partial states. The help adapter should render reference data from the catalog while keeping the existing authored corpus and TUI.

At this phase, Cobra becomes a compatibility interpreter rather than the owner of help facts.

4.3.5 Phase 4 - Migrate first-party applications

Migrate repositories with increasing complexity:

1. small static command trees;
2. commands with reusable sections and config/env sources;
3. aliases and dynamic commands;
4. plugins and wrapper commands;
5. complex help corpora and completions.

Every migration contributes fixtures to the compatibility corpus. Differences are triaged centrally so the same edge case is not rediscovered in every repository.

4.3.6 Phase 5 - Isolate the Cobra adapter

Move Cobra integration to a separate module or package with a clear support window. Core Glazed no longer imports Cobra or pflag. Applications that integrate with an existing Cobra root can depend on the adapter intentionally.

4.3.7 Phase 6 - Publish the native major release

Remove compatibility shims that distort the native model, publish the final grammar and migration guide, and require explicit opt-in packages for legacy Cobra integration.

4.4 Compatibility profiles

A single Boolean “Cobra-compatible” is too coarse. Define a profile:

```
type CompatibilityProfile struct {
    InterspersedOptions    bool
    ShortClusters          bool
    AttachedShortValues    bool
    LongEquals             bool
    BooleanNegation        NegationPolicy
    ScalarRepeat           RepeatPolicy
    ListWithinSource       ListOccurrencePolicy
    UnknownOptions         UnknownOptionPolicy
    CommandPrefixMatching  bool
    CaseSensitiveCommands  bool
    NormalizeUnderscores   bool
    HelpOnBareNamespace    bool
}
```

A named profile can be serialized in the manifest:

```
{
  "syntaxProfile": "glazed-cli/v1",
  "features": {
    "interspersedOptions": true,
    "shortClusters": true,
    "longEquals": true,
    "commandPrefixMatching": false
  }
}
```

This makes syntax part of the public contract and allows future profiles without hidden global switches.

4.4.1 Worked example: a deliberate break

Suppose current pflag accepts multi-character ShortFlag strings in some Glazed definitions, while conventional short options are one rune. The compiler can classify existing declarations:

- one rune: preserve;
- empty: preserve;
- multiple runes used as a single-dash name: unspecified or legacy;
- multiple runes intended as a cluster: modeling error.

A major release may reject multi-rune short names and require a long binding. That is an intentional break with a compiler diagnostic and automatic migration suggestion.

4.4.2 Worked example: command-prefix matching

Prefix matching appears convenient:

```
golem n s query
```

But adding command sync under note can make s ambiguous. The accepted language changes when an unrelated command is added. The native profile should disable prefix matching by default. If a legacy application enables it, the parser must diagnose ambiguity rather than select by order.

4.5 Differential testing as the migration gate

A differential harness should compare semantic outcomes, not only strings.

```
type Outcome struct {
    CommandID CommandID
    Values     CanonicalValues
    Error      *DiagnosticClass
    Stdout     []byte
    Stderr     []byte
    ExitCode   int
    HelpModel  *CanonicalHelp
    Completions []CanonicalCandidate
}
```

For each fixture:

```
old := runWithCobra(fixture)
new := runWithNative(fixture)
diff := compareCanonical(old, new)
classify(diff)
```

Text rendering is compared separately because the help modernization may intentionally change layout while preserving facts.

4.5.1 Canonicalization rules

- command identity uses stable command ID, not display path alone;
- values are compared after semantic decoding;
- maps are sorted;
- secret values are replaced with presence markers;
- error strings are mapped to classes where exact legacy text is not contractual;
- help is normalized into commands, fields, defaults, choices, and prose blocks;
- completion candidates are compared as typed sets plus ordering policy.

4.5.2 Evidence gates

A phase may advance when:

1. all preserve fixtures are equivalent;
2. every difference has an approved compatibility class;
3. no fixture leaks secrets;
4. race and fuzz tests pass;
5. first-party application smoke tests pass;
6. performance stays within declared budgets;
7. the migration guide includes every intentional break.

4.6 Fuzzing and property testing

The parser is a high-value fuzz target because token interactions are combinatorial.

4.6.1 Catalog fuzzing

Generate authored command sets with bounded sizes. Assert:

- compiler never panics;
- diagnostics are deterministic;
- successful catalogs satisfy binding injectivity;
- manifest round trips preserve public facts;
- compiling a normalized manifest is idempotent where supported.

4.6.2 Invocation fuzzing

For a fixed catalog:

- arbitrary UTF-8 token sequences never panic or loop;
- parser terminates in $O(n + m)$ expected time, where n is token count and m is cluster rune count;
- every occurrence field exists in the selected effective schema;
- redacted traces contain no secret raw values;
- parsing after `--` never interprets an option.

4.6.3 Generative valid programs

A generator can derive valid invocations from the catalog. This creates a stronger oracle than arbitrary fuzzing:

```
generate command path
choose optional inherited/local options
render each occurrence in a legal surface form
choose legal positional values
parse
assert selected command and decoded assignment equal generated semantics
```

Equivalent renderings create metamorphic pairs.

4.7 Rearchitecting the Command framework

Removing Cobra creates an opportunity to clarify the Command model, but a rewrite should be conservative. Four layers should be explicit.

4.7.1 Layer 1 - authoring declarations

Convenient constructors, options, YAML loading, plugins, aliases, and reusable sections. This layer may be mutable during assembly.

4.7.2 Layer 2 - compiled static contract

Stable IDs, effective schemas, bindings, contracts, indexes, source locations, and help references. This layer is immutable and frontend-neutral.

4.7.3 Layer 3 - invocation interpretation

Parse result, source resolution, provenance, validation, diagnostics, and concrete execution plan. This layer is per invocation.

4.7.4 Layer 4 - effects

Existing bare, writer, and structured-row execution plus future effect adapters. This layer owns resources and returns errors.

The current framework sometimes mixes layers. `CommandDescription` carries authoring and static-contract facts. Cobra mounting mutates parser state. Alias descriptions clone schemas at runtime. Help reads some facts through Cobra. The compiler boundary lets these concerns be separated without forcing the authoring API to become verbose.

4.7.5 Proposed top-level API

```
type Application struct {
    Catalog catalog.CommandCatalog
    Parser   cli.Parser
    Resolver sources.Resolver
    Help     help.Service
    Runner   runner.Runner
}
```

```
func BuildApplication(
    commandSet cmds.CommandSet,
    opts ...ApplicationOption,
) (*Application, []Diagnostic)
```

```
func (a *Application) Execute(
    ctx context.Context,
    request ExecuteRequest,
) ExecuteResult
```

`ExecuteResult` should carry error, exit recommendation, selected command, resolved values, and optional diagnostics. The library should not call `os.Exit`; the executable adapter decides the process exit code.

4.8 Command sets and catalogs

Current command construction often mounts commands one at a time. The native compiler benefits from an explicit collection.

```
type CommandSet interface {
    Commands(context.Context) ([]cmds.Command, error)
    Aliases(context.Context) ([]alias.CommandAlias, error)
    HelpSections(context.Context) ([]help.Section, error)
    SourceIdentity() SourceIdentity
}
```

A `CommandSet` can be static, plugin-backed, YAML-backed, or composed. Compilation receives the entire set and can therefore validate cross-command collisions and aliases atomically.

Definition 4.3 (catalog generation). A catalog generation is one immutable compiled snapshot with a generation ID and source fingerprint.

Dynamic applications can compile a new generation and atomically swap it for future invocations. In-flight invocations retain the generation they began with. This is safer than mutating a live command tree during completion or help rendering.

4.8.1 Worked example: plugin reload

```
generation 41: commands A, B, plugin X
compile generation 42 in background: commands A, B, plugin Y
validate all collisions and help links
atomic pointer swap to generation 42
new invocations use 42
in-flight invocation on X finishes against 41
release 41 when no readers remain
```

This epoch model generalizes beyond Cobra removal and supports reliable dynamic command discovery.

4.9 Framework-injected inputs

Glazed injects structured-output fields for commands that support row output. Other applications inject logging, profiling, or server controls. Injection must be a compile step, not an afterthought.

```
type FieldContributor interface {
    Contributions(ctx CompileContext, command StaticCommand) ([]FieldContribution,
        ↪ error)
}
```

Contributors run in a deterministic order and declare source identity. The compiler merges all contributions and reports collisions with both claims. A contributor can be conditional on command contract:

```
if command.Output.Mode == rows:
    inject output.format
    inject output.output-fields
    inject output.max-output-rows
```

Help, manifests, completion, and parsing see the same injected fields because they all consume the final catalog.

4.10 Constraints beyond requiredness

Cobra's flag groups demonstrate that relationships should be declared once and interpreted by validation and completion. Glazed can generalize this idea at the semantic field level.

```
type Constraint interface {
    ID() ConstraintID
    Fields() []FieldID
    Validate(ResolvedView) *Diagnostic
    Narrow(PartialResolvedView) ConstraintHints
    Describe() ConstraintDescription
}
```

Built-ins:

- AllOrNone(fields...);
- AtLeastOne(fields...);
- AtMostOne(fields...);
- ExactlyOne(fields...);

- `Requires(a,b);`
- `Conflicts(a,b);`
- numeric/string ranges;
- conditional requiredness based on a semantic value.

The same constraint feeds:

- runtime validation;
- completion narrowing;
- help prose;
- form generation;
- JSON Schema export where expressible;
- tests.

JSON Schema provides a useful interchange vocabulary for many structural constraints, though not every CLI or effect invariant maps exactly ([JSON Schema Project 2022](#)). The catalog can export an approximation with explicit notes about unsupported semantics.

4.10.1 Worked example: cache clear

Constraint:

```
ExactlyOne(cache.all, cache.key).
```

If neither is resolved, completion can suggest both. After `--all`, completion hides `--key`. Runtime rejects both or neither. Help renders “exactly one required.” The declaration is singular; interpreters differ.

4.11 Modern help: from command template to help service

A modern help system should answer different questions without overloading one `--help` page.

4.11.1 help - what can I do?

Human-oriented reference plus authored explanation:

```
golem help note search
golem note search --help
golem help --query 'structured output'
```

4.11.2 describe - what is the static contract?

Machine-oriented catalog output:

```
golem describe note search --format json
golem describe --all --format jsonl
```

Includes stable IDs, syntax profile, fields, bindings, constraints, output contract, aliases, examples, and catalog fingerprint. It excludes resolved values and secrets.

4.11.3 explain - why did this invocation resolve this way?

Provenance and interpretation:

```
golem explain note search query --limit 5
```

Shows command resolution, alias expansion, sources, shadowed values, defaults, deprecations, and validation decisions. It performs parsing and source loading but no command effect.

4.11.4 plan - what would execute?

Value-dependent plan:

```
golem plan cache clear --all
```

Shows effect class, selected runner, output pipeline, potential resources, and warnings. It must remain distinct from execution and state clearly which checks are advisory.

Fundamentals - static versus dynamic semantics. describe is static semantics: facts derivable from the catalog. explain and plan are dynamic interpretations of a concrete input and environment. Mixing them causes help to load secrets, fail on unavailable config, or accidentally initialize effects.

4.12 Help rendering modes

A single HelpDocument can support several views.

4.12.1 Concise terminal view

Optimized for one screen:

Search indexed notes and emit structured rows.

Usage:

```
golem note search QUERY [options]
```

Arguments:

QUERY	Search expression. Required.
-------	------------------------------

Common options:

-n, --limit INT	Maximum results. Default: 20.
-f, --format VALUE	table json jsonl csv tsv. Default: table.
-t, --tag TEXT	Restrict to a tag. Repeatable.

Global options:

-c, --config TEXT	Configuration file.
-v, --verbose	Verbose diagnostics.

Examples:

```
golem note search 'prefix cuts' --format jsonl
```

More:

```
golem help note search --long
golem explain note search ...
```

4.12.2 Long contextual view

Adds authored description, source precedence, constraints, field provenance names, output contract, deprecations, and related tutorials.

4.12.3 Searchable TUI

Reuse the current Bubble Tea interface but feed it graph nodes and exact catalog facts. Search filters can include command ID, field ID, topic, package, section type, and effect class.

4.12.4 JSON view

Stable schemas for agents and editors. This should not be terminal Markdown wrapped in JSON.

4.12.5 Web documentation

Build from the same help graph. Use collision-free paths based on stable command IDs or escaped path segments, not ambiguous flattened filenames.

4.13 Contextual help and diagnostics

The help service can subscribe to structured diagnostics:

```
type HelpService interface {
    ForCommand(CommandID, HelpOptions) HelpDocument
    ForDiagnostic(Diagnostic, Context) HelpDocument
    Search(Query) SearchResult
}
```

For `E_UNKNOWN_FLAG`, it can show suggestions and the option section. For `E_REQUIRED`, it can show all sources that may satisfy the field. For `E_CHOICE`, it can show choice descriptions. For a constraint failure, it can show corrected forms.

The service must not imply that an unavailable external value will work. Dynamic completers and planners may fail; help should distinguish static facts from current observations.

4.14 Help content governance

Model-derived help cannot rot structurally, but authored prose can. Add validation:

- every linked command ID exists;
- every linked field ID exists in at least one catalog generation or is marked historical;
- examples parse under their declared syntax profile;
- examples identified as safe may be executed in controlled tests;
- deprecated names point to replacements;
- topics and package names belong to known vocabularies;
- generated web routes are injective;
- secret values do not appear in fixtures.

Examples that parse the product's own command language create executable documentation. A safe subset can be tested without running effects by invoking `plan` or `parse-only` modes.

4.15 Shell completion protocol

Cobra's hidden completion command is a strong pattern worth preserving conceptually: the executable queries its own live model. The native protocol should be versioned and structured.

Request:

```
{
  "protocol": "glazed-completion/v1",
  "argv": ["note", "search", "x", "--fo"],
  "cursor": {"token": 4, "byte": 4},
  "shell": "zsh"
}
```

Response:

```
{
  "replace": {"token": 4, "startByte": 0, "endByte": 4},
  "candidates": [
    {
      "value": "--format",
      "display": "--format VALUE",
      "description": "output framing",
      "kind": "option",
      "appendSpace": true
    }
  ],
  "fallback": "none"
}
```

A line-oriented compatibility mode can remain for shell scripts, but internal semantics should not be encoded only as tab-separated strings and a final integer.

4.15.1 Completion safety policy

- pass cancellation and a short deadline;
- prohibit stdout logging inside the protocol path;
- classify completers as static, filesystem, local service, or remote;
- cache by catalog fingerprint and safe resolved inputs;
- redact secrets from queries and logs;
- return partial candidates with diagnostics when a dynamic source fails;
- never perform the command's effect.

4.16 Error and exit semantics

The current move to RunE corrected an important boundary: library execution errors should return. A native application API should make this explicit.

```
type ExecuteResult struct {
  Command      CommandID
  Values       *values.Values
  Diagnostics  []Diagnostic
  Err          error
  SuggestedExit int
}
```

The executable adapter decides:

```
result := app.Execute(ctx, request)
renderDiagnostics(os.Stderr, result.Diagnostics)
if result.Err != nil {
  os.Exit(result.SuggestedExit)
}
```

The library does not call `os.Exit`, print unconditionally, or terminate before deferred application cleanup runs.

4.16.1 Exit taxonomy

A suggested mapping, configurable at the application boundary:

Class	Exit
success	0
invocation/validation error	2
not found or unavailable command	2
command effect failure	1
cancellation / signal	130 or platform policy
internal invariant violation	70 or application policy

Exact codes are less important than separating parse failure from effect failure and keeping policy outside the core runner.

4.17 Secrets and security boundaries

A semantic catalog is discoverable by design. It is not an authorization system. The following distinctions must remain explicit:

- a command exists versus a caller may execute it;
- a field exists versus its value may be disclosed;
- completion suggests a resource versus the caller is authorized to use it;
- a manifest describes an effect versus the effect is safe;
- an alias resolves versus its target is permitted.

4.17.1 Secret field policy

For a secret field:

- help may show name, purpose, accepted source classes, and whether it is required;
- manifests omit defaults and example values;
- provenance may show `source=env`, `present=true` but not raw text;
- diagnostics refer to type/presence without echoing the value;
- completion must not propose secret values;
- debug traces redact before serialization;
- explain requires an explicit reveal policy and should default to redaction.

4.17.2 Authorization

Authorization belongs to a trusted interpreter before effects. The catalog can declare required capabilities, but matching the declaration is not sufficient evidence that checks were performed. A future execution plan may include:

```
type CapabilityRequirement struct {
    Name      string
    Resource ResolverRef
    Action    string
}
```

The runner or host checks these requirements against current identity. Help can describe them without granting them.

4.18 Downstream compatibility and semantic versioning

Removing Cobra affects three audiences.

4.18.1 Application authors using only Glazed APIs

The migration should preserve constructors and command interfaces where possible. They should gain compiler diagnostics and a native frontend with minimal changes.

4.18.2 Application authors importing Cobra through Glazed

Some code may receive or mutate `*cobra.Command`, register Cobra-only flags, or depend on Cobra hooks. This code cannot be made frontend-neutral automatically. The compatibility module and migration tooling should identify:

- raw Cobra imports;
- `*cobra.Command` parameters;
- direct pflag access;
- Cobra lifecycle hooks;
- custom help/usage templates;
- direct completion registration;
- command mutations after application build.

The existing linter work is a foundation for this inventory.

4.18.3 End users and scripts

They depend on syntax, output, diagnostics, and completion. The major release must publish:

- grammar differences;
- changed error text/classes;
- help layout changes;
- deprecation timelines;
- alias behavior;
- exit status policy;
- shell completion installation changes.

A compatibility checker can run a script corpus against old and new binaries and produce a report.

4.19 Migration tooling

Useful tools include:

4.19.1 `glazed doctor cli`

Compiles the command set and reports:

- collisions currently hidden by mount order;
- multi-rune short flags;
- aliases with ambiguous precedence;
- fields whose semantic and CLI identities are conflated;
- help links using unstable strings;
- raw Cobra dependencies;
- secret defaults in public metadata;
- dynamic mutations after build.

4.19.2 `glazed manifest diff`

Compares two catalog manifests:

breaking:

- removed command `note.export`
- changed field `search.limit` type `integer` -> `string`
- removed long binding `--output`

compatible:

- added alias `--format` for `output.format`
- added optional field `search.case-sensitive`

documentation:

- changed short description of `note.search`

4.19.3 glazed cli replay

Runs fixture corpora against two frontends or binaries, classifies differences, and emits machine-readable reports.

4.19.4 glazed help lint

Validates semantic links and parses examples.

4.20 Governance of the native language

Once Glazed owns a parser, syntax changes become framework governance. Adopt a small language-evolution process:

1. every new syntax feature has a grammar note and ambiguity analysis;
2. every feature has parser, completion, help, and manifest semantics;
3. every feature specifies interaction with `--`, aliases, positionals, and source precedence;
4. compatibility fixtures cover both acceptance and rejection;
5. the syntax profile version changes when accepted programs or interpretations change materially;
6. global behavior switches are avoided; policy is catalog- or command-scoped;
7. undocumented parser recovery is not treated as a contract until specified.

This process prevents the native parser from accumulating the same implicit behavior the migration is trying to escape.

4.21 Future research

Several extensions become possible after the catalog is native.

4.21.1 Formal grammar extraction

Generate a machine grammar for each command and use derivative-based or parser-combinator techniques to prove static completion soundness for the finite portion of the language (Ford 2004; Might, Darais, and Spiewak 2011).

4.21.2 Constraint solving

General constraints can narrow completion and generate forms. A finite-domain solver could interpret choices, implications, and exclusivity without mutating field definitions.

4.21.3 Incremental catalog compilation

Large plugin systems may recompile only affected subtrees while preserving a generation fingerprint. The challenge is keeping effective inherited scopes and alias dependencies coherent.

4.21.4 Editor and agent protocols

A catalog resembles a language-server schema. A local server could provide completion, hover help, diagnostics, semantic tokens, and command planning to editors. The protocol should transport stable IDs and spans rather than terminal strings.

4.21.5 Effect contracts and policy

Static effect declarations can feed confirmation UIs, audit planning, sandbox selection, and authorization. Research is needed to distinguish declared intent from enforced evidence.

4.21.6 Documentation verification

Examples can be compiled, parsed, planned, and in safe cases executed against fixtures. A documentation build can fail when a command or field link disappears.

4.21.7 Reproducible command manifests

Catalog fingerprints could become release artifacts. A binary could expose the fingerprint of its embedded command language, allowing scripts and agents to verify they are speaking to the expected contract.

4.22 Final conclusions

The core finding is architectural rather than syntactic:

Glazed should not replace Cobra with a new flag loop. It should compile its own command language and make every frontend an interpreter over that compiled catalog.

The pinned source already contains the semantic foundation: command descriptions, sectioned typed schemas, field codecs, multi-source middleware, provenance-rich values, aliases, a frontend-neutral runner, an authored help corpus, and a modern TUI. Cobra fills a real gap, but the gap is now well defined: whole-program catalog compilation, command and option grammar, partial parsing, completion protocol, and reference-help projection.

The experimental prototype demonstrates that these pieces can be implemented without Cobra and can operate at ordinary CLI performance scales. It also exposes the work that remains: integrate all Glazed field types, define compatibility profiles, preserve source semantics, build a real help IR, implement dynamic completion safely, carry precise source maps through aliases, and validate the design across downstream repositories.

The migration should proceed through evidence gates. First compile the existing model. Then dual-run parsers. Then move completion and help to the catalog. Then migrate applications. Only after the compatibility corpus is explained should Cobra leave the core module.

A successful result is more than dependency reduction. It gives Glazed an explicit language architecture: one semantic model, one compiler, several interpreters, explainable values, structured diagnostics, modern help, and a stable contract for humans and tools.

4.23 Exercises for Chapter 4

1. Classify five observed Cobra behaviors into preserve, repair, intentional break, or unspecified. Justify each classification.
2. Design the schema of one compatibility fixture that covers config, env, alias injection, CLI override, and a structured-output field.
3. Propose a canonical comparison for help that allows layout modernization while detecting factual drift.
4. Write pseudocode for dual-running parsers without executing a command twice.
5. Define a catalog-generation swap protocol safe for concurrent parsing and completion.
6. Design a `FieldContributor` for logging fields. How does it avoid collisions with application-owned `--verbose`?
7. Express `ExactlyOne(all, key)` as a Boolean formula. Derive completion hints for the states none set, all set, and both set.
8. Decide which syntax-profile fields are command-scoped and which must be application-wide.
9. Specify a redaction test for explain when a secret value is supplied through CLI, config, and environment in separate fixtures.
10. Design glazed manifest diff rules for renaming a long binding while preserving semantic field ID.
11. State a deprecation timeline for the Cobra compatibility module.
12. Explain why a searchable help TUI is not a substitute for concise `--help` output.
13. Give one static fact appropriate for describe, one provenance fact appropriate for explain, and one effect-dependent fact appropriate for plan.
14. Design a shell completion deadline and partial-failure policy for a remote completer.
15. Propose three release metrics that would indicate the native migration is safe to make default.
16. Identify one claim in this thesis that requires downstream repository evidence before it can be considered established.

Selected Solutions and Discussion

4.24 Solution 1.1: semantic versus frontend concerns

A choice set is semantic because every frontend must enforce the same domain, although its display is frontend-specific. A short flag and terminal width are frontend-specific. A default is semantic resolution policy. An environment prefix is an environment binding. A file-loading field type is mixed: the semantic result type and validation are shared, while path syntax and access policy may be frontend or host specific. A dangerous-effect warning is semantic contract metadata rendered by a frontend.

4.25 Solution 1.4: two format fields

Use distinct identities such as `input.format` and `output.format`. Bind them to `--input-format` and `--format` or `--output-format`. Configuration can use `input.format` and `output.format`. Help links use identities, not display strings.

4.26 Solution 2.3: why injectivity is necessary

If $B_v(-v) = \text{logging.verbose}$ and $B_v(-v) = \text{output.version}$, the token `-v` has two legal transitions from the same state. The scanner relation is no longer a function. Selection by registration order makes language semantics depend on construction order, violating determinism.

4.27 Solution 2.7: override is not commutative

Let $A(x) = 1$ and $B(x) = 2$. Then $(A \triangleright B)(x) = 2$, while $(B \triangleright A)(x) = 1$. Source order is therefore semantic and must be recorded.

4.28 Solution 2.11: completion after --

For `golem cache clear --`, the parser is in operand-only mode. Static option names must not be returned. If the command has no positional fields, completion may return no candidates or a diagnostic hint that no further operands are accepted. Shell file fallback should be explicit rather than accidental.

4.29 Solution 3.5: alias source maps

Represent each rewritten token with an origin:

```
type TokenOrigin struct {  
    Kind      string // user, alias-prefix, generated
```

```
UserToken  int
AliasID    string
AliasToken int
}
```

A diagnostic on an injected token can state that alias find supplied `--format=jsonl` for field `output.format`, while still pointing the user to the alias invocation.

4.30 Solution 3.10: help with missing required fields

Invoke the control path before command admission. Parse enough to identify the command and help option, then return a `HelpRequest` rather than running source resolution and required validation. An integration test should define a required field, execute command `--help`, assert success, assert no config access if not needed, and assert the handler did not run.

4.31 Solution 4.4: dual-run without duplicate effects

Both frontends produce canonical parse and resolution outcomes. Compare them before execution. Select exactly one outcome according to the feature gate, then build one execution plan and call the runner once. Never run both command handlers.

4.32 Solution 4.7: exactly-one formula

For Boolean presence variables a and k :

$$(a \vee k) \wedge \neg(a \wedge k).$$

With neither set, completion suggests both as alternatives. With a set, it suppresses k . With both set, it may offer removal fix-its rather than additional candidates.

4.33 Solution 4.10: binding rename

If semantic field ID and type are unchanged, adding new binding `--format` while retaining deprecated `--output` is compatible. Removing `--output` is a breaking syntax change. The manifest diff should distinguish semantic compatibility from CLI-binding compatibility.

Glossary and Notation

Alias macro. A command-path rewrite that may inject fixed tokens before caller tokens. It is more expressive than an alternate name.

Arity. The number of raw values consumed by one option occurrence. Boolean flags often have implicit arity zero at the token level but produce one semantic raw value.

Authored model. Mutable command, field, alias, and help declarations assembled from Go, YAML, plugins, or options before compilation.

Binding. A frontend-specific way to address a semantic field, such as `--limit`, `GOLEM_LIMIT`, or config path `search.limit`.

Binding injectivity. The law that one visible frontend name maps to at most one semantic field in an effective command scope.

Catalog. The immutable, normalized, validated representation of the complete command language.

Catalog fingerprint. A deterministic digest of a normalized public command contract.

Catalog generation. One immutable catalog snapshot used consistently by an invocation, help request, or completion query.

CLI source. The source assignment derived from parsed command-line occurrences. It is one input to multi-source resolution.

Command contract. Static metadata describing output shape, effect class, cancellation, idempotency expectations, and other public promises.

Command declaration. Frontend-neutral description of a command path, schema, prose, kind, effect, and metadata.

Command path language. The finite set of declared command paths represented as a trie.

Command trie. A rooted prefix tree in which edges are command names and nodes are declared commands or structural namespaces.

Compatibility corpus. Versioned fixtures recording current successful and failing invocations, source assignments, help facts, completion, errors, and exits.

Compatibility profile. Explicit configuration of token grammar and parser policies, such as clusters, interspersed options, repeat behavior, and normalization.

Compilation. Whole-program normalization, indexing, validation, alias resolution, effective-scope calculation, and freezing.

Completion candidate. A typed proposal for extending a partial invocation, including replacement span, kind, description, and shell behavior.

Completion soundness. The property that a static completion candidate can extend the current prefix without an immediate unknown-name error.

Conceptual help. Authored tutorials, rationale, workflows, caveats, and examples.

Constraint. A relation over one or more semantic fields interpreted by validation, completion, help, and possibly forms or schema export.

Control path. A non-effect path such as help, version, describe, explain, or completion that must not trigger ordinary command admission.

Decode. Convert raw source text into a typed semantic value under a field codec.

Describe. Produce the static machine-readable contract of commands and fields from the catalog.

Diagnostic. Structured compile, parse, decode, resolve, validate, or execute information with a stable code and contextual data.

Differential testing. Run two implementations on the same input and compare canonical outcomes to discover behavioral differences.

Effective field signature. The set of semantic fields visible at a command after inherited and local contributions are compiled.

Effective binding relation. The mapping from frontend names visible at one command to semantic field IDs.

Effect interpreter. The runner that consumes a validated execution plan and performs command work.

Explain. Interpret a concrete invocation and show command resolution, source precedence, provenance, defaults, and validation without running the effect.

Field codec. A partial function from raw values to one semantic type, including validity checks.

Field contributor. A compiler extension that injects semantic fields based on command contracts.

Field identity. Stable semantic coordinate for a field, independent of CLI spelling.

Field signature. Finite typed declaration of fields accepted by a command.

Frontend. An encoding and interaction boundary such as CLI, environment, config file, JSON, HTTP, editor protocol, or form.

Help document IR. Renderer-neutral typed document built from catalog facts and authored help nodes.

Help graph. Graph linking commands, fields, examples, tutorials, concepts, packages, and external references by stable identities.

Inherited field. A field declared in an ancestor scope and included in descendant effective schemas.

Interspersed options. Policy allowing options after positional operands or between command tokens where unambiguous.

Invocation. One token sequence plus external source environment interpreted against one catalog generation.

Invocation path. The command tokens typed by the user, which may name an alias.

Macro expansion provenance. Mapping from rewritten tokens to user and alias-source origins.

Manifest. Versioned machine-readable projection of the public command catalog.

Metamorphic test. A test that transforms input in a semantics-preserving way and checks equal outcomes.

Occurrence. One raw binding of a frontend form to a semantic field, with position and syntactic form.

Operand. A token treated as positional input rather than an option or command name.

Option terminator. `--`, after which CLI tokens are operands rather than options.

Override. Right-biased combination of partial maps in which the later source wins where defined.

Partial parse. Parser mode that accepts an incomplete final token and returns residual state for completion.

Plan. Value-dependent, non-executing description of what a concrete invocation would run.

Positional grammar. Ordered declaration of required, optional, and variadic operands.

Presence. Whether a source supplied a field, separate from whether its value is empty or null.

Provenance log. Ordered record of source contributions and transformations leading to a resolved value.

Raw occurrence multimap. Map from semantic field IDs to ordered raw textual occurrences produced by scanning.

Reference help. Exact model-derived command, input, default, choice, constraint, alias, and output facts.

Repeat policy. Handling of repeated occurrences of a scalar option: reject, first wins, or last wins.

Required-after-merge. Requiredness law evaluated on the final resolved assignment after all configured sources.

Residual language. $D_w(L) = \{x \mid wx \in L\}$, the legal suffix language after prefix w .

Resolve. Merge decoded source assignments according to precedence and field merge policy, preserving provenance.

Right-biased override. Partial-map operator $A \triangleright B$ in which values from B replace values from A when both are defined.

Scanner state. Current command node, token index, mode, option-termination state, occurrences, operands, and optional trace.

Semantic field. Typed input slot with stable identity, constraints, default, and metadata independent of frontend names.

Semantic model. Frontend-neutral commands, fields, constraints, values, and effects.

Source assignment. Partial map from semantic field IDs to raw or typed values supplied by one source.

Source precedence. Ordered policy determining how defaults, config files, environment, provided values, aliases, and CLI combine.

Structural namespace. Non-runnable command trie node used to organize child commands.

Syntax profile. Versioned public description of accepted CLI grammar and policies.

Trace. Optional ordered record of parser or resolver transitions, subject to secret redaction.

Transitional adapter. Compatibility layer that projects the native catalog into Cobra while applications migrate.

Typed command language. Command and field grammar whose accepted tokens are interpreted into typed semantic assignments and effects.

Whole-program validation. Checks requiring the complete command set, such as binding collisions and alias targets.

Appendix A: Prototype API and Reproduction

The source bundle contains:

```
prototype/  
  go.mod  
  nativecli.go  
  nativecli_test.go  
  benchmark_test.go  
  cmd/demo/main.go  
  
experiments/  
  validation.txt  
  benchmark.txt  
  demo-resolution.json  
  manifest.json  
  audit_existing_string_parser.py  
  audit_existing_string_parser.tsv
```

Reproduce:

```
cd prototype  
go test ./... -race -cover -count=1  
go vet ./...  
go test -bench=. -benchmem -run '^$' -count=5 ./...  
go run ./cmd/demo
```

The prototype is research code. It is not a proposed drop-in Glazed package and has no compatibility guarantee.

Appendix B: Migration Checklist

4.34 Semantic inventory

- ☐ Enumerate every command source and dynamic loading path.
- ☐ Assign stable command IDs.
- ☐ Assign stable field IDs separate from CLI names.
- ☐ Inventory all section prefixes and inherited fields.
- ☐ Inventory aliases and fixed injected values.
- ☐ Inventory command kinds and output contracts.
- ☐ Inventory current raw Cobra/pflag dependencies.

4.35 Grammar

- ☐ Specify long-option forms.
- ☐ Specify short-option and cluster forms.
- ☐ Specify Boolean negation.
- ☐ Specify -- exactly.
- ☐ Specify interspersed-option policy.
- ☐ Specify repeat policy.
- ☐ Specify list splitting and cross-source merge.
- ☐ Specify unknown-option forwarding.
- ☐ Specify command aliases and prefix matching.

4.36 Compiler

- ☐ Validate command and alias collisions.
- ☐ Validate field ID collisions.
- ☐ Validate effective long and short binding injectivity.
- ☐ Validate positional grammar.
- ☐ Validate constraints and defaults.
- ☐ Validate help references.
- ☐ Produce deterministic diagnostics and manifest.
- ☐ Redact secrets.

4.37 Runtime

- ☐ Parse to semantic field occurrences.
- ☐ Preserve original token spans.
- ☐ Merge all sources before required validation.
- ☐ Preserve provenance.
- ☐ Return errors; do not exit inside the library.

- ☐ Keep help/completion/describe effect-free.
- ☐ Support cancellation.

4.38 Evidence

- ☐ Build differential fixture corpus.
- ☐ Add metamorphic tests.
- ☐ Add fuzz tests.
- ☐ Add downstream application tests.
- ☐ Benchmark real catalogs.
- ☐ Review every difference.
- ☐ Publish migration guide and syntax profile.

Appendix C: Compatibility Questions That Must Be Answered

1. Are options recognized before, between, and after subcommands?
2. Does first positional input stop option recognition by default?
3. Are short options exactly one rune?
4. Are Boolean short options clusterable?
5. Can valued short options use attached text?
6. Is `--name=value` supported for every non-Boolean field?
7. Are `--bool=false` and `--no-bool` both accepted?
8. How are repeated scalars handled?
9. How are repeated lists handled within one source?
10. How do lists merge across sources?
11. Does `--` stop command resolution as well as option parsing?
12. Can unknown options be forwarded, and under what boundary?
13. Are underscores normalized to hyphens?
14. Are command and option names case-sensitive?
15. Are unique command prefixes accepted?
16. How are aliases expanded relative to caller tokens?
17. Can aliases target aliases?
18. Which sources may satisfy a required field?
19. Does help bypass source loading and validation?
20. What are parse, validation, effect, and cancellation exit codes?
21. What completion protocol is stable?
22. Which help facts are generated and which are authored?
23. How are secrets represented in manifests and traces?
24. When is a dynamic catalog generation considered ready to publish?

Appendix D: Glazed Source Evidence Map

All source links below are pinned to commit `a2bff0ece5f46b90975d7687f7c0dca2ea516d22`.

- [pkg/cmds/cmds.go](#) - CommandDescription, command interfaces, and frontend-neutral effect shapes.
- [pkg/cmds/schema/schema.go](#) - ordered sections, schema composition, and Cobra-specific contamination in the core schema boundary.
- [pkg/cmds/fields/definitions.go](#) - field definitions, names, short flags, types, defaults, choices, requiredness, and positional roles.
- [pkg/cmds/fields/field-type.go](#) - semantic field type vocabulary.
- [pkg/cmds/fields/parse.go](#) - semantic decoding of raw values.
- [pkg/cmds/fields/cobra.go](#) - field-to-pflag mapping, Cobra usage, and collection of changed values.
- [pkg/cmds/fields/strings.go](#) - existing Cobra-free string parser seed and the source of the binding-key counterexample.
- [pkg/cmds/fields/gather-arguments.go](#) - ordered positional binding and final variadic list behavior.
- [pkg/cmds/fields/field-value.go](#) - values, parse-step histories, merging, and defaults.
- [pkg/cmds/values/section-values.go](#) - sectioned resolved values and decoding.
- [pkg/cmds/sources/middlewares.go](#) - source middleware composition.
- [pkg/cmds/sources/cobra.go](#) - Cobra and positional-argument sources.
- [pkg/cmds/runner/run.go](#) - frontend-neutral command execution.
- [pkg/cmds/alias/alias.go](#) - aliases, targets, fixed flags/arguments, schema cloning, and delegated effects.
- [pkg/cli/cobra.go](#) - building and mounting Cobra commands and executing through RunE.
- [pkg/cli/cobra-parser.go](#) - source ordering and the Cobra parser adapter.
- [pkg/cli/cobra_error_test.go](#) - error-propagation tests.
- [pkg/help/help.go](#) - help-section store and query.
- [pkg/help/model/section.go](#) - authored help section schema.
- [pkg/help/render.go](#) - Markdown templates and Glamour terminal rendering.
- [pkg/help/ui/model.go](#) - searchable Bubble Tea help browser.
- [pkg/help/cmd/cobra.go](#) - deep Cobra dependency in command help adaptation.
- [README.md](#) - current public architecture and reduced structured-output surface.
- [go.mod](#) - direct Cobra and pflag dependencies in the pinned snapshot.

Bibliography

- Claessen, Koen, and John Hughes. 2000. "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs." In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–79. <https://doi.org/10.1145/351240.351266>.
- Ford, Bryan. 2004. "Parsing Expression Grammars: A Recognition-Based Syntactic Foundation." In *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 111–22. ACM. <https://doi.org/10.1145/964001.964011>.
- JSON Schema Project. 2022. "JSON Schema: A Media Type for Describing JSON Documents, Draft 2020-12." <https://json-schema.org/draft/2020-12/json-schema-core>.
- McKeeman, William M. 1998. "Differential Testing for Software." *Digital Technical Journal* 10 (1): 100–107.
- Microsoft. 2024. "Language Server Protocol Specification, Version 3.17." <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>.
- Might, Matthew, David Darais, and Daniel Spiewak. 2011. "Parsing with Derivatives: A Functional Pearl." In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming*, 189–95. ACM. <https://doi.org/10.1145/2034773.2034801>.
- Owens, Scott, John Reppy, and Aaron Turon. 2009. "Regular-Expression Derivatives Re-Examined." *Journal of Functional Programming* 19 (2): 173–90. <https://doi.org/10.1017/S0956796808007090>.
- The Open Group. 2024a. "Getopt Utility." POSIX.1-2024. <https://pubs.opengroup.org/onlinepubs/9799919799/utilities/getopt.html>.
- . 2024b. "Utility Syntax Guidelines." POSIX.1-2024, Base Definitions, Chapter 12. https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap12.html.